akeyless-watchover

Akeyless governance as code

User and reference manual

RBAC and path design · YAML model · Terraform delivery · CLI and API · brownfield export · verification

Version 1.4.0

October 29, 2026

Repository: `https://github.com/MisterErre/akeyless-watchover`

Contents

I Getting started **1**

1 Introduction **2**

1.1 What akeyless-watchover is . . . 2
1.2 Scope . . . 2
1.3 Who this manual is for . . . 2
1.4 How this manual is organised . . . 2
1.5 Conventions . . . 3

2 Installation and prerequisites **4**

2.1 Prerequisites . . . 4
2.2 Get the kit . . . 4
2.3 Install the skill . . . 4
2.4 Create your governance repository . . . 5
2.5 Rebuilding this manual . . . 5

3 Quick start: from a diagram to a delivered configuration **6**

3.1 Step 1 – Describe the organisation to Claude . . . 6
3.2 Step 2 – Put the model in the repository . . . 6
3.3 Step 3 – Test offline . . . 7
3.4 Step 4 – Bootstrap the CI identity (once) . . . 7
3.5 Step 5 – Merge request, plan, apply . . . 7
3.6 Step 6 – Prove the result live . . . 7
3.7 Step 7 – Add notifications . . . 8

4 Everyday workflows **9**

4.1 Grant access . . . 9
4.2 Revoke access or offboard a team . . . 9
4.3 Onboard a new application or business unit . . . 9
4.4 Onboard a workload identity . . . 9
4.5 Emergency change (break-glass) . . . 10
4.6 Temporary and just-in-time access . . . 10
4.7 Rename a role or an auth method . . . 10

4.8 Bring existing configuration under Terraform . . . 10
4.9 Migrate from CyberArk Conjur . . . 11
4.10 Set up certificate issuance . . . 11
4.11 Add or adopt an event forwarder . . . 11
4.12 Harden the account . . . 11
4.13 Gateway administration and audit-log forwarding . . . 11
4.14 Handle a drift finding . . . 11

II Concepts and design **12**

5 Architecture **13**
5.1 Components . . . 13
5.2 Identities . . . 13
5.3 What runs where . . . 13
5.4 Change flow . . . 15

6 The Akeyless authorization model **16**
6.1 Objects and namespaces . . . 16
6.2 Identity . . . 16
6.3 Access Roles . . . 16
6.4 Anti-escalation . . . 17
6.5 Gateway access - the second layer . . . 17
6.6 Inspection . . . 18

7 From a structural diagram to Akeyless configuration **19**
7.1 Method (always produce these five artifacts) . . . 19
7.2 Path rules . . . 19
7.3 Naming of governance objects . . . 19
7.4 Role catalogue . . . 19
7.5 Worked example . . . 20

III Reference **22**

8 Skill commands (talking to Claude) **23**
8.1 `help` . . . 23
8.2 `design` . . . 24
8.3 `change` . . . 24
8.4 `review` . . . 25

8.5 pipeline . . . 25
8.6 import . . . 26
8.7 conjur . . . 26
8.8 pki . . . 27
8.9 harden . . . 27
8.10 gateway . . . 28
8.11 verify . . . 28
8.12 cli . . . 29
8.13 explain . . . 29

9 Kit commands (scripts/watchover) 31
9.1 watchover test . . . 31
9.2 watchover simulate . . . 33
9.3 watchover verify . . . 35
9.4 watchover export . . . 36
9.5 watchover forwarder . . . 41
9.6 watchover conjur . . . 44
9.7 watchover account-settings . . . 46
9.8 watchover token . . . 47
9.9 watchover build-skill . . . 48
9.10 watchover help . . . 48
9.11 watchover version . . . 51

10 YAML model reference 52
10.1 roles/<any>.yaml - access roles (one file per BU, protected by CODEOWNERS) . . . 52
10.2 auth_methods.yaml . . . 52
10.3 folders.yaml . . . 53
10.4 governance.yaml . . . 53
10.5 pki.yaml . . . 55
10.6 gateway.yaml (needs var.gateway_api) . . . 55
10.7 access_expectations.yaml . . . 56

11 Terraform reference implementation (terraform/) 57
11.1 Inputs . . . 57
11.2 Guards (fail the plan, never the apply) . . . 57
11.3 Provider facts this code relies on (verified in v2.0.5 source/schema) . . . 58

12 Terraform variables and outputs **60**
12.1 Governance root module (`terraform/`) . . . 60
12.2 Bootstrap module (`bootstrap/`) . . . 61

13 CLI playbook **63**
13.1 RBAC . . . 63
13.2 Auth methods . . . 63
13.3 Inspection . . . 64
13.4 Account settings (CLI-only part) . . . 64
13.5 Gateway . . . 64
13.6 PKI . . . 64

14 Akeyless REST API or CLI? **65**
14.1 Short answer . . . 65
14.2 Evidence . . . 65
14.3 Differences that do matter . . . 65
14.4 Decision rules . . . 66
14.5 Did the CLI unlock more exports? . . . 66
14.6 In this kit . . . 67

IV Operations **68**

15 Pipeline: bootstrap, CI/CD, drift and state **69**
15.1 1. Bootstrap (once, by an Akeyless admin) . . . 69
15.2 2. CI templates (`ci/`) . . . 69
15.3 3. State . . . 70
15.4 4. Drift . . . 70

16 Testing and verification **71**
16.1 Offline access simulator . . . 71
16.2 Live verification . . . 71
16.3 Running everything locally . . . 72

17 Export existing configuration and translate it into Terraform **73**
17.1 What is exported . . . 73
17.2 Transport: CLI or REST . . . 74
17.3 Two modes . . . 74
17.4 Filters . . . 75

17.5 Secrets . . . 75
17.6 Procedure . . . 75

18 CyberArk Conjur -> Akeyless 77
18.1 Translation rules . . . 77

19 Certificates / PKI governance 79

20 Security considerations 80
20.1 Identities . . . 80
20.2 Review . . . 80
20.3 Secrets and state . . . 80
20.4 Guards and least privilege . . . 81
20.5 Working with Claude . . . 81
20.6 The demo back end . . . 81

21 Troubleshooting and FAQ 82
21.1 Guard failures (plan stops with “Resource precondition failed”) . . . 82
21.2 Pipeline and Terraform . . . 83
21.3 Kit commands . . . 83
21.4 Frequently asked questions . . . 84

22 Maintaining and extending the kit 85
22.1 Updating the kit in your governance repository . . . 85
22.2 Upgrading the Terraform provider . . . 85
22.3 Versioning and releases . . . 85
22.4 Adding a kit command . . . 85
22.5 Adding a guard . . . 86
22.6 Maintaining this manual . . . 86

23 Limitations, workarounds and open verifications 87
23.1 Not possible (platform or provider 2.0.5) . . . 87
23.2 Not verified against a live account (marked VERIFY) . . . 87

A Exit codes 89

B Environment variables 90

C Repository layout 91

D Glossary 92

E Sources **93**

Part I

Getting started

Chapter 1

Introduction

1.1 What akeyless-watchover is

akeyless-watchover turns the *governance* of an Akeyless account into code that is reviewed, tested and delivered from one Git repository: one repository, one change, one configuration delivered. It has two parts that work together:

The skill
: A Claude skill (`SKILL.md` plus reference documents). In a conversation, Claude translates an organisation diagram or a plain-language access request into Akeyless paths, roles, auth methods and associations, reviews changes, writes exact CLI commands and explains Akeyless authorization precisely. Chapter 8 lists its commands.

The kit
: A tested repository template: a YAML model, generic Terraform that delivers it with the `akeyless-community/akeyless` provider, a one-time CI identity bootstrap, CI templates for GitHub and GitLab, and the `scripts/watchover` tool (offline tests, an access simulator, live verification, export of existing configuration to Terraform, Conjur policy translation, event-forwarder scaffolding and more). Chapter 9 lists its commands.

1.2 Scope

In scope is the configuration *inside* Akeyless: folders and paths, access roles and their rules, auth methods and their bindings, management-plane permissions, account settings, custom fields, key policies, event forwarders, PKI issuers and CA keys, and the Gateway's administrative plane (allowed access, log forwarding, Conjur migration). Out of scope are deploying Gateways and integrating consumers (applications, Kubernetes injectors, CI secret retrieval).

1.3 Who this manual is for

- **PAM and platform engineers** who own the Akeyless account: read everything, start with Chapter 3.
- **Business-unit engineers** who request or approve access: Chapter 4 (recipes), the skill commands `change` and `verify`, and the model reference.
- **Reviewers and auditors**: the RBAC model (Chapter 6), testing and verification, security considerations (Chapter 20) and the limitations.

1.4 How this manual is organised

Part I, Getting started
: installation, a guided first delivery, and recipes for everyday tasks.

Part II, Concepts and design
the architecture, how Akeyless authorization works, and how to derive paths and roles from an organisation diagram.

Part III, Reference
every skill command and kit command with *all* options and an example execution, every YAML key, the Terraform implementation, its variables and outputs, the CLI playbook, and how to choose between the REST API and the CLI.

Part IV, Operations
pipeline, testing, brownfield import, Conjur migration, PKI, security, troubleshooting, maintenance and the known limitations.

Appendices
exit codes, environment variables, repository layout, glossary and sources.

1.5 Conventions

- `monospace` is a command, path, option or YAML key. `<value>` is a placeholder; `[x]` is optional; `a|b` is a choice.
- **VERIFY** marks a statement that no primary source confirms. All of them are collected in Chapter 23; check them on your account before relying on them.
- Commands are run from the root of your governance repository unless a section says otherwise.

1.5.1 How the example executions were produced

Every example in a grey terminal box was *executed* by `docs/manual/build.py` when this PDF was built, and the output is shown unedited except for long outputs, where the middle is cut and marked `[... N lines omitted ...]`, and temporary paths, which are replaced by `..`. The exit code is shown under each box.

> **Offline demo back end** No Akeyless account is used to build the manual. Kit commands that call Akeyless (`verify`, `export`, `account-settings`, `token`) ran against a stand-in: `docs/manual/demo/akeyless` plays the CLI and answers from the same fixtures the test-suite uses, and a local HTTP endpoint plays the REST API. For `verify`, the stand-in evaluates a copy of the model to which one manual console change was added, so the example shows what a real drift finding looks like. On your account the commands are identical; only the data differs. Where the command shown differs from the one executed (a production URL instead of the local endpoint), the box says so.

The *example sessions* of the skill commands (talking to Claude) are illustrative: Claude's answers depend on the conversation. They show the shape of the output, abridged.

Chapter 2

Installation and prerequisites

2.1 Prerequisites

Component	Version	Needed for
Python	3.10 or newer, with `pyyaml`	every kit command
`pytest`	any recent	`watchover test` (Python tests)
Terraform *or* OpenTofu	Terraform $\geq$ 1.7, OpenTofu $\geq$ 1.8 (mocked-provider tests)	`watchover test`, plan and apply
Provider `akeyless-community/akeyless`	`~> 2.0` (verified: 2.0.5)	downloaded by `init`
Akeyless CLI (`akeyless`)	current	`verify`, `export --transport cli`, `account-settings`
`curl`, `jq`	any	`watchover token`
`zip`	any	`watchover build-skill`
Bash	4 or newer	the shell helpers and test scripts
pandoc, latexmk, XeLaTeX	pandoc $\geq$ 2.11, TeX Live	rebuilding this manual only

Commands marked *offline* in Chapter 9 never contact Akeyless. The others need a CLI profile or a token of an identity with enough rights; see each command.

2.2 Get the kit

```
git clone https://github.com/MisterErre/akeyless-watchover.git
cd akeyless-watchover
python3 -m pip install pyyaml pytest      # add --user or a virtualenv as you prefer
scripts/watchover version
scripts/watchover test --tool tofu        # or --tool terraform; everything must pass
```

2.3 Install the skill

Build the skill package (CI also attaches it to every run as the artefact `akeyless-watchover-skill`):

```
scripts/watchover build-skill             # -> dist/akeyless-watchover.zip
```

The ZIP contains `SKILL.md`, `references/` (the Markdown documents of `docs/`) and `template/` (the tested kit).

Claude (web and desktop). Upload the ZIP under **Customize > Skills** (older interface: **Settings > Capabilities > Skills > Upload skill**). Code execution must be enabled.

Claude Code. Unzip into your personal or project skills folder:

```
unzip dist/akeyless-watchover.zip -d ~/.claude/skills/            # personal
unzip dist/akeyless-watchover.zip -d <project>/.claude/skills/    # project
```

Claude loads the skill when you talk about Akeyless governance, or explicitly with `/akeyless-watchover <command>`. Check the installation with `/akeyless-watchover help`.

2.4 Create your governance repository

The kit is a template: copy it into the repository that will own your Akeyless configuration.

1. Create an (internal or private) repository, e.g. `akeyless-governance`.
2. Copy `terraform/`, `bootstrap/`, `scripts/` and the CI template for your platform (`ci/github/akeyless-governance.yml` to `.github/workflows/`, or `ci/gitlab/.gitlab-ci.yml` to the root). Copy `ci/CODEOWNERS.example` to `CODEOWNERS` and adapt the owners.
3. Add a remote state backend to `terraform/versions.tf` (and a separate one for `bootstrap/`). The state contains secrets: encrypted, versioned, access-restricted (Chapter 15).
4. Replace the example model in `terraform/model/` with yours (Chapter 3 shows how), keeping `access_expectations.yaml` in step.
5. Run `scripts/watchover test` and commit.

2.5 Rebuilding this manual

```
python3 docs/manual/build.py                 # regenerate, execute the examples, compile the PDF
python3 docs/manual/build.py --check         # are the generated reference parts current?
```

Section 22.6 explains what is generated and from which source.

Chapter 3

Quick start: from a diagram to a delivered configuration

This chapter walks through a first delivery end to end with the example organisation that ships with the kit: a *payments* business unit with a *checkout* application in `prod` and `nonprod`, human access through Entra ID (OIDC) and workloads on AWS and Kubernetes. Replace the names with yours as you go.

3.1 Step 1 – Describe the organisation to Claude

Design from a description

You `/akeyless-watchover design` Payments BU, apps checkout and refunds, environments prod and nonprod. Payments admins (Entra group grp-akl-payments-admins) manage nonprod and read prod. Checkout prod pods (EKS namespace payments, service account payments-api) read their own prod secrets. Auditors list everything. Break-glass secrets are under prod/break-glass and nobody but the platform team reads them.
Claude Returns five artefacts: (1) the path tree `/payments/<app>/<env>/...`; (2) an access matrix persona × path; (3) the YAML model (`model/roles/payments.yaml`, `model/auth_methods.yaml`, `model/folders.yaml`); (4) `access_expectations.yaml` entries that prove the matrix; (5) what the plan must show, and the equivalent CLI commands.

3.2 Step 2 – Put the model in the repository

A role file is short. Rules are path patterns with capabilities; associations bind the role to an auth method, narrowed by sub-claims:

```
roles:
  /governance/roles/payments/team-admin:
    description: Payments engineers manage nonprod secrets of all their apps, read prod
    audit_access: scoped
    rules:
      - { path: "/payments/+/nonprod/*", capabilities: [read, list, create, update, delete] }
      - { path: "/payments/+/prod/*", capabilities: [read, list] }
      - { path: "/payments/+/prod/break-glass/*", capabilities: [deny] }
    associations:
      - auth_method: /governance/auth/sso-entra-oidc
        sub_claims:
          groups: [grp-akl-payments-admins]
```

Every key is documented in Chapter 10. The expectations file states, per persona, the exact capabilities on sample paths (`[]` means no access):

```
checks:
  - { persona: payments-admin, path: /payments/checkout/prod/db/app,           expect: [read, list
    ] }
  - { persona: payments-admin, path: /payments/checkout/prod/break-glass/root, expect: [] }
```

3.3 Step 3 – Test offline

```
scripts/watchover simulate --model terraform/model \
    --expectations terraform/model/access_expectations.yaml --explain
scripts/watchover test --tool tofu
```

`simulate` evaluates the expectations against the model (wildcards, deny precedence, sub-claims); `test` runs everything CI runs, including the Terraform guards that reject unsafe models (a root-scope rule, a deny mixed with other capabilities, an association without sub-claims to an auth method that forces them, …). Nothing touches Akeyless.

3.4 Step 4 – Bootstrap the CI identity (once)

An Akeyless administrator creates the identity the pipeline runs as. It is keyless: the CI's own OIDC token is trusted, scoped to your repository (plan) and to its protected main branch (apply).

```
cd bootstrap
cp bootstrap.tfvars.example bootstrap.tfvars          # platform, repository, gitlab_url...
export AKEYLESS_ACCESS_ID=... AKEYLESS_ACCESS_KEY=... # admin API key, used once
terraform init && terraform apply -var-file=bootstrap.tfvars
terraform output pipeline_access_id                    # -> CI variable
    AKEYLESS_PIPELINE_ACCESS_ID
```

Then create the CI variables: `AKEYLESS_PIPELINE_ACCESS_ID`, and as masked JSON maps `TF_VAR_auth_method_secrets`, `TF_VAR_event_forwarder_secrets`, `TF_VAR_gateway_secrets`. Optional: `AKEYLESS_API`, `AKEYLESS_GATEWAY_API`.

3.5 Step 5 – Merge request, plan, apply

1. Push a branch and open a merge request. CI runs the checks and a `plan` with the plan identity. CODEOWNERS asks the business unit and the platform team to approve.
2. Read the plan. Creations and in-place updates are expected; *replacements* of roles or auth methods are red flags (names are ForceNew; an auth method replacement changes its access ID). Ask Claude: `/akeyless-watchover review <paste the plan>`.
3. Merge. The apply job runs behind a protected environment, then applies the account settings the provider cannot manage (`watchover account-settings`).

3.6 Step 6 – Prove the result live

Give each test persona a CLI profile (`akeyless configure --profile <name> ...`), add `profile: <name>` to it in `access_expectations.yaml`, and run:

```
scripts/watchover verify --expectations terraform/model/access_expectations.yaml
```

The nightly drift job then keeps the account equal to Git: a console change makes the plan non-empty and the job fails; the next apply reverts it.

3.7 Step 7 – Add notifications

```
scripts/watchover forwarder --name /governance/events/secops-mail --type email \
    --email-to secops@example.com --preset critical --items-source '/*' --auth-method-source '/*' \

    --write terraform/model/governance.yaml
```

Commit, merge request, merge: the forwarder is delivered like everything else.

> **Already have an Akeyless account full of configuration?** Start with `watchover export` instead of Step 2 (Chapter 17): it writes the model and the import blocks, and the first plan should show imports only.

Chapter 4

Everyday workflows

Each recipe gives the goal, the steps, and what to check. All changes follow the same delivery: model change → `watchover test` → merge request (checks, plan, CODEOWNERS approval) → merge (apply) → optional `watchover verify`.

4.1 Grant access

1. Ask Claude for the smallest diff: `/akeyless-watchover change give fraud-analysts read on /payments/+/prod/*`.
2. Apply the diff to `terraform/model/roles/<bu>.yaml` and add the new checks to `access_expectations.yaml` – including at least one *negative* check (what must stay denied).
3. `scripts/watchover simulate ... --explain` shows which rule grants what.

Check: the plan changes one role (rules in place) and maybe adds one association; nothing is replaced.

4.2 Revoke access or offboard a team

Remove the rule or the association (or narrow the sub-claims) and turn the corresponding expectation into `expect: []`. Offboarding a person needs no change here: human access follows IdP group membership through sub-claims, so remove the person from the IdP group. **Check:** the plan removes exactly what you intended; role deletions are blocked when `delete_protection` is on – turn it off in one merge request, delete in the next.

4.3 Onboard a new application or business unit

1. `/akeyless-watchover design <the new BU/app, environments and personas>`.
2. New business unit: a new `model/roles/<bu>.yaml` and a CODEOWNERS line for it. New application in an existing BU: usually no role change at all, because rules use `/<bu>/+/<env>/*`; add folders and expectations.
3. Add the folders you want to protect (`delete_protection`) to `model/folders.yaml`.

4.4 Onboard a workload identity

Add the auth method (AWS IAM, Azure AD, Kubernetes, OAuth2/JWT, Universal Identity, …) to `model/auth_methods.yaml` with the narrowest binding (account, role, namespace, service account), associate it with a workload role, and add a check. Secrets such as a client secret are referenced with `*_var` keys and supplied through `TF_VAR_auth_method_secrets`. Kubernetes and

LDAP methods also need Gateway-side configuration, which is out of scope. **Check:** hand the `auth_method_access_ids` output to the consumer team; it is not secret.

4.5 Emergency change (break-glass)

When production is down and the pipeline cannot wait: make the change with the CLI as an administrator (`/akeyless-watchover cli <task>` gives the exact command), then *immediately* raise the merge request that codifies it. If you skip the merge request, the nightly drift job fails and the next apply reverts your change (role rules are authoritative).

4.6 Temporary and just-in-time access

Temporary rules (`--ttl`) exist only in the CLI and API; the next apply on that role removes them. For just-in-time access, model the *possibility* of access in Git and let each grant be requested, approved and audited in Akeyless: for Secure Remote Access, `sra-rule` with `request_access` for requesters and `approval_authority` on a separate approver role (the two cannot be combined in one rule; Chapter 6). How access requests work for other object types depends on your Akeyless edition – ask `/akeyless-watchover explain access requests` and check the result against your account.

4.7 Rename a role or an auth method

Names are ForceNew in the provider: changing the YAML key alone destroys and recreates the object (for an auth method this issues a *new access ID* and breaks its consumers). To rename a role without replacement:

1. Rename it in Akeyless: `akeyless update-role --name <old> --new-name <new>`.
2. Rename the YAML key, remove the old addresses from state (`terraform state rm 'akeyless_role.this["<old>"]'` and its associations) and add `import` blocks for the new ones (roles by name, associations by `"<role>:<auth method>"`).
3. The plan must show imports only.

Do the same for an auth method only if its access ID must not change; otherwise plan the consumer cut-over first.

4.8 Bring existing configuration under Terraform

1. `scripts/watchover export --out export/ --profile admin` (or `--transport api`). Use `--types`, `--only` or the persona filter to start small.
2. `/akeyless-watchover import export/REPORT.md`: Claude resolves every report item.
3. Move the model files and `imports.tf` into `terraform/`; iterate until the plan shows *imports only*; apply. For objects outside the model use `--hcl --generate`.

Details: Chapter 17.

4.9 Migrate from CyberArk Conjur

Secrets move with an Akeyless Gateway migration (`model/gateway.yaml`); RBAC is translated by `scripts/watchover conjur`, which writes roles and a bindings file in which you choose the auth method and sub-claims for each Conjur host and user. Chapter 18.

4.10 Set up certificate issuance

`/akeyless-watchover pki <requirements>` designs CA keys and issuers in `model/pki.yaml`, the requester roles and an expiry notification. Chapter 19.

4.11 Add or adopt an event forwarder

Akeyless cannot list event forwarders, so they cannot be exported. Create new ones with `scripts/watchover forwarder ... --write terraform/model/governance.yaml`. To bring an existing one under Terraform, recreate its definition with the same command plus `--import-existing` (import ID = its name), then check the first plan shows an import and no unexpected update. Put the printed secret keys into `TF_VAR_event_forwarder_secrets`.

4.12 Harden the account

`/akeyless-watchover harden` proposes `account_settings` (the part the provider manages), `account_settings_cli` (bound IPs, client types, locks; applied by `watchover account-settings` in the apply job), mandatory custom fields and key policies. Always run `watchover account-settings --dry-run` first: a wrong `bound-ips` can lock out your own pipeline.

4.13 Gateway administration and audit-log forwarding

`model/gateway.yaml` defines who may administer the Gateway (allowed access) and where it forwards audit logs (syslog or Splunk). Set `gateway_api`; use one state or workspace per Gateway cluster.

4.14 Handle a drift finding

The nightly job failed: someone changed Akeyless outside Git. Read the plan output.

- The change was wrong: run the apply job (or wait for the next merge) – it reverts it.
- The change was needed: codify it in a merge request; the plan becomes empty after merge.

`watchover verify` finds the same class of problem from the access side (see the example in Section 9.3).

Part II

Concepts and design

Chapter 5

Architecture

5.1 Components

5.2 Identities

Identity	Created by	Bound to	Can
Account admin (human)	Akeyless account	-	run `bootstrap/` once; break-glass
CI auth method `/governance/auth/ci-terraform` (OAuth2/JWT)	`bootstrap/`	CI OIDC issuer (GitHub or GitLab)	authenticate pipelines, nothing by itself
Role `.../terraform-plan`	`bootstrap/`	CI auth method, sub-claim = repository	read RBAC/auth config, list items, read `/pki/*`
Role `.../terraform-apply`	`bootstrap/`	CI auth method, sub-claims = repository **and** protected main branch	administer everything it manages (admin-equivalent, because Akeyless forbids granting what you do not hold)
Everything in `terraform/model`	main pipeline	as modelled	as modelled

The CI identity lives in a separate state (`bootstrap/`), so the main pipeline can never destroy its own access.

5.3 What runs where

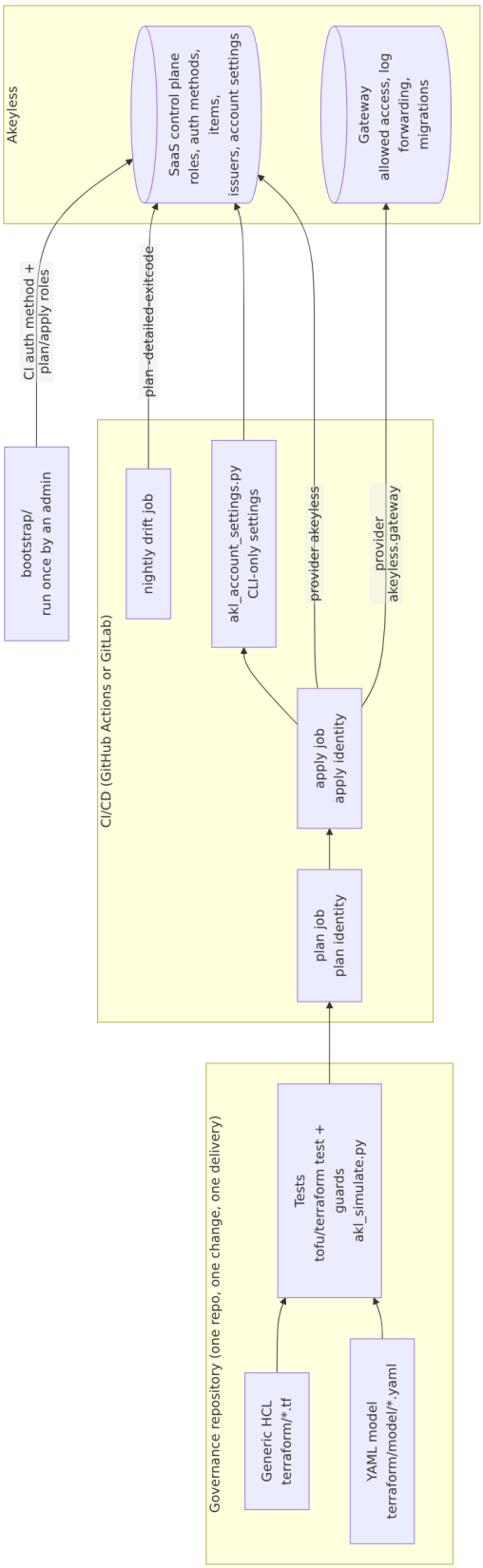


Figure 5.1: Diagram from docs/architecture.md

Object family	API endpoint	Terraform provider
Roles, rules, associations, auth methods, folders, issuers, keys, custom fields, key policies, event forwarders, account settings	`https://api.akeyless.io` (or a Gateway's `/api/v2`, which proxies the API)	`akeyless`
Gateway allowed access, log forwarding, migrations	the Gateway API (`https://<gw>:8000/api/v2` or `:8081`)	`akeyless.gateway` (alias)
Account settings not exposed by the provider	REST `/auth` + CLI `update-account-settings`	- (`scripts/akl_account_settings.py`)

One Gateway cluster per state: for several clusters, run the same code with one workspace or var-file per cluster (`gateway_api` and `model/gateway.yaml` differ).

5.4 Change flow

1. Engineer edits YAML (normally only `model/roles/<bu>.yaml`) and opens a merge request.
2. `checks`: format, validate, `terraform test` (20+ cases, mocked provider), guard-message proof, offline access expectations (`akl_simulate.py`), script tests.
3. `plan` with the plan identity; reviewers read the plan (CODEOWNERS: BU + platform).
4. Merge -> `apply` (protected environment, manual approval) -> CLI-only account settings.
5. Optional: `akl_verify.py` with test personas proves effective access live.
6. Nightly `drift` fails when someone changed Akeyless outside Git; the next apply reverts it (role rules are authoritative).

Chapter 6

The Akeyless authorization model

Verified against docs.akeyless.io (RBAC, Sub-Claims, CLI reference), the Akeyless OpenAPI and the Terraform provider v2.0.5. Items marked **VERIFY** are not confirmed by a primary source.

6.1 Objects and namespaces

- **Items** (static/rotated/dynamic secrets, keys, certificates, PKI/SSH issuers), **Targets**, **Access Roles** and **Auth Methods** live in separate trees, each named by a slash path.
- Item folders are implicit: creating `/a/b/c` creates `/a` and `/a/b`. `folder create` / `akeyless_folder` only adds empty folders, description, tags and delete protection.

6.2 Identity

- A caller authenticates through an **Auth Method** (API key, OIDC, SAML, LDAP, OAuth2/JWT, AWS IAM, Azure AD, GCP, K8s, certificate, Universal Identity, ...). The token carries the claims of that login.
- There are **no RBAC user or group objects**. Human scoping comes from IdP claims (e.g. `groups`).
- Akeyless **Groups** (`create-group`, `akeyless_group`) bundle auth methods for Akeyless **OIDC applications** - they are not RBAC groups.

6.3 Access Roles

- **Default deny**: no association, no access. The built-in admin role has full access.
- A role = **rules** + **management-plane settings**; a role is bound to auth methods by **associations** that may carry **sub-claims**.

6.3.1 Rules

rule_type	governs	capabilities
`item-rule` (default)	items	`read, list, create, update, delete, deny`
`target-rule`	targets	same
`role-rule`	access roles	same
`auth-method-rule`	auth methods	same
`sra-rule`	Secure Remote Access	`allow_access, request_access, justify_access_only, approval_authority, upload_files, download_files`
`ara-rule`	Agentic Runtime Authority	`allow_access` (API/CLI; not documented by provider 2.0.5 - VERIFY)

- Every capability implies `list`.
- `deny` overrides all other permissions on the matched path (docs). Prove cross-role behaviour with Impersonate As / `akl_verify.py`. `deny` does **not** apply to SRA paths.
- `approval_authority` cannot be combined with `request_access`.
- Wildcards: `*` at the end (`/payments/*`) or as a prefix (`/payments/devops-*`); `+` = any characters within exactly one segment (`/payments/+/prod/*`).
- Templated paths from sub-claims, e.g. `/{{groups}}/{{email}}/*` (the docs show both `{{x}}` and `{x}` - VERIFY on your account).
- Temporary rules: `set-role-rule --ttl <minutes>` (CLI/API only).

6.3.2 Sub-claims (on associations)

- Within one key, comma-separated values = OR; different keys = AND.
- Values accept `*` and `?` globs.
- Keys and values are case-sensitive unless the association sets `case_sensitive=false`.
- Gateway >= 4.19 accepts explicit `AND` / `OR` inside one key's value, evaluated left to right.
- `force_sub_claims=true` on an auth method makes every association to it require sub-claims.
- `akeyless describe-sub-claims` prints the claims of the current token.

6.3.3 Management-plane settings (role attributes)

Attribute	Values
`audit_access`	none, own, scoped, all
`analytics_access`	none, own, all
`gw_analytics_access`, `sra_reports_access`, `event_center_access`, `isi_access`, `unlock_secrets`, `approve_access_request`	none, scoped, all
`usage_reports_access`, `event_forwarders_access`	none, all (+ `event_forwarders_name`)
`reverse_rbac_access`	scoped, all

"scoped" = objects the caller can list. Internally these are stored as special rules (`search-rule`, `reports-rule`, ... with paths `/*`, `/self`, `/scoped`).

6.4 Anti-escalation

- A non-admin can only create or change roles that grant capabilities they themselves hold on the path.
- A non-admin creator's own `deny` rules are copied onto roles they create. Terraform exposes them as read-only `restricted_rules` and never removes them.
- Consequence: the identity that applies RBAC must hold everything it grants (admin-equivalent).

6.5 Gateway access - the second layer

RBAC decides which **objects** a caller can touch. **Gateway Allowed Access** decides who may administer a given Gateway and which features (`defaults`, `targets`, `classic_keys`,

`automatic_migration`, `ldap_auth`, `dynamic_secret`, `k8s_auth`, `log_forwarding`, `zero_knowledge_encryption`, `rotated_secret`, `caching`, `event_forwarding`, `admin`, `kmip`, `general`, `rotate_secret_value`). Creating rotated/dynamic secrets on a Gateway requires both layers.

6.6 Inspection

- `akeyless describe-permissions --path P --type item|am|role|target` - the **caller's own** permissions on P (`client_permissions`).
- `akeyless reverse-rbac --path P --type item|am|role|target|sra|ara` - which auth methods/roles reach P.
- Console: account menu -> **Impersonate As** (auth method + sub-claims).

Chapter 7

From a structural diagram to Akeyless configuration

7.1 Method (always produce these five artifacts)

1. **Inventory** from the diagram: tenants/BUs, applications, environments, data classes (PCI, break-glass), personas (workload, dev team, platform admin, auditor, SecOps, certificate requester) and the identity source of each persona.
2. **Path tree** for items.
3. **Access matrix**: personas x path scopes -> capabilities (+ rule type).
4. **YAML model diff** (`terraform/model/...`) and **access expectations** (`access_expectations.yaml`).
5. **Plan expectations** and the equivalent **CLI** for break-glass/bootstrap.

Ask before guessing when the diagram does not say who the identity source is, how environments are segregated, who may change RBAC, or who owns break-glass.

7.2 Path rules

- Delegation boundary first, then the dimensions you grant on, most specific last: `/<bu>/<app>/<env>/<class>/<secret>`, e.g. `/payments/checkout/prod/db/orders-ro`.
- If environment is the primary security boundary (separate prod operators): `/<env>/<bu>/<app>/...`. Cross-cut the other dimension with + (`/payments/+/prod/*`).
- Reserve top-level namespaces: `/shared`, `/pki/{ca,issuers,certs}`, `/platform`.
- Only auditors (list) and admins get `/*`; the guards enforce it (`allow_root_scope`).
- Stable identifiers only (app IDs, not team or person names): renames destroy and recreate objects in Terraform and break consumer paths.
- Carve-outs (`break-glass`, `pci`) as a fixed segment so one `deny` covers them.

7.3 Naming of governance objects

Object	Pattern (enforced by `var.naming`)
Roles	`/governance/roles/<bu>/<role>`
Auth methods	`/governance/auth/<idp-or-platform>-<scope>`
Targets	`/targets/<bu>/<app>/<env>/<name>`
Issuers / CA keys	`/pki/issuers/<name>`, `/pki/ca/<name>`
Event forwarders	`/governance/events/<name>`

7.4 Role catalogue

Role	Rules	Binding
`<bu>/<app>-<env>-workload`	item read,list on `/<bu>/<app>/<env>/*`; deny on carve-outs	cloud/K8s auth method bound to the workload
`<bu>/team-admin`	item CRUD on nonprod, read,list on prod, deny carve-outs	SSO + `groups=<idp-group>`
`<bu>/delegated-admin` (optional)	item CRUD on `/<bu>/*`; target CRUD on `/targets/<bu>/*`	SSO group; RBAC changes stay in the pipeline
`platform/auditors`	item list `/*`; role/auth-method read,list `/*`; `audit_access=all`	SSO group
`platform/pki-requesters-<x>`	rule on the issuer + its `destination_path`	workload or SSO
`platform/breakglass`	admin-equivalent	MFA SSO group, alert on use

Admin plane: only the pipeline identity holds create/update/delete on `role-rule` and `auth-method-rule`. Humans get read,list on those plus `audit_access`.

7.5 Worked example

Diagram: *"Payments BU owns apps checkout and refunds, each with prod and nonprod. Payments engineers (Entra group grp-akl-payments-admins) manage nonprod and read prod. Checkout prod pods (namespace payments, SA payments-api) and the checkout Lambda role (payments-app, account 123456789012) read checkout prod. Nobody but break-glass may read prod/break-glass. Auditors see everything but no values."*

Path tree:

```
/payments/checkout/{prod,nonprod}/...
/payments/refunds/{prod,nonprod}/...
/payments/+/prod/break-glass/...   (carve-out)
```

Access matrix:

Persona	`/payments/+/nonprod/*`	`/payments/checkout/prod/*`	`/payments/+/prod/break-glass/*`	roles/ auth config
payments-engineer	CRUD	read	**deny**	-
checkout-prod workload	-	read	**deny**	-
auditor	list	list	list	read

Model: exactly `terraform/model/roles/payments.yaml` and `platform.yaml` in this repository; the expectations in `terraform/model/access_expectations.yaml` encode the matrix and run on every PR.

Part III

Reference

Chapter 8

Skill commands (talking to Claude)

These commands are typed in a conversation with Claude once the skill is installed (Claude Code: `/akeyless-watchover <command> [args]`; Claude web/desktop: `/akeyless-watchover ...` or simply describe the task – the skill also triggers without a command). Arguments are free text. Every section ends with an *example session*: the prompt and an abridged version of the answer. Answers depend on your model and conversation, so the sessions are illustrative; the artefacts they produce are the files and commands documented in Part III.

Command	Purpose
`help`	Show this guide, or the details of one skill command.
`design`	Translate an organisation/structure diagram into Akeyless configuration.
`change`	Change existing access with the smallest possible model diff.
`review`	Review a proposed change for least privilege, guard violations and blast radius.
`pipeline`	Set up the governance repository: bootstrap identity, CI, CODEOWNERS, state.
`import`	Export existing config (RBAC, auth methods incl. OIDC, folders, custom fields, key policies, PKI issuers, account settings) and translate it into Terraform.
`conjur`	Migrate from CyberArk Conjur: secrets via the Gateway, RBAC via translation.
`pki`	Design private/public CA issuers and who may request certificates.
`harden`	Account-level hardening.
`gateway`	Gateway admin plane and audit-log forwarding.
`verify`	Prove who can access what.
`cli`	Exact Akeyless CLI commands for a task (break-glass, bootstrap, one-off checks).
`explain`	Explain an Akeyless concept precisely (sub-claims, deny, wildcards, anti-escalation...).

8.1 `help`

Syntax

```
/akeyless-watchover help [command]
```

Arguments

[command]

Purpose

Show this guide, or the details of one skill command.

Delivers

The command table, parameters and examples.

Example

```
/akeyless-watchover help design
```

Example session

You /akeyless-watchover help change
Claude Shows the `change` entry: arguments (*an access request in plain words*), what it delivers (a minimal diff of `model/roles/<bu>.yaml`, new expectations, who must approve) and an example. Without an argument: the table of all skill commands and the kit commands.

8.2 `design`

Syntax

`/akeyless-watchover design <diagram | description of BUs, apps, environments, personas>`

Arguments

<diagram | description of BUs, apps, environments, personas>

Purpose

Translate an organisation/structure diagram into Akeyless configuration.

Delivers

Path tree, access matrix, YAML model (roles, auth methods, folders), access expectations, plan expectations, CLI equivalent.

Example

`/akeyless-watchover design <paste or attach the diagram>`

Example session

You /akeyless-watchover design *[attached diagram: BU Payments with apps checkout, refunds; prod and nonprod; Entra groups; EKS workloads]*

Claude **1. Path tree** `/payments/{checkout,refunds}/{prod,nonprod}/...`, `/payments/<app>/prod/break-glass/...`

2. Access matrix payments-admin: nonprod CRUD, prod read; checkout-prod-pod: own prod read; auditor: list everything; everyone else: nothing.

3. Model `model/roles/payments.yaml` (`team-admin`, `checkout-prod-workload`, …), `model/auth_methods.yaml` (OIDC with `force_sub_claims`, K8s bound to namespace `payments`), `model/folders.yaml`.

4. Expectations 11 checks including the negative ones (break-glass, other BUs).

5. Plan expectations and CLI "+ 2 roles, + 2 auth methods, + 3 associations, no replacements"; the equivalent `akeyless create-role / set-role-rule / assoc-role-am` commands.

8.3 `change`

Syntax

`/akeyless-watchover change <access request in plain words>`

Arguments

<access request in plain words>

Purpose

Change existing access with the smallest possible model diff.

Delivers

Diff of model/roles/<bu>.yaml (and auth_methods.yaml if needed), new expectations, who must approve (CODEOWNERS).

Example

`/akeyless-watchover change give fraud-analysts read on /payments/+/prod/*`

Example session

You /akeyless-watchover change give the fraud-analysts Entra group read on refunds prod
Claude Smallest diff – a new role, because no existing role has the right audience:

```
+  /governance/roles/payments/fraud-analysts:
+    description: Fraud analysts read refunds prod
+    rules:
+      - { path: "/payments/refunds/prod/*", capabilities: [read, list] }
+      - { path: "/payments/refunds/prod/break-glass/*", capabilities: [deny] }
+    associations:
+      - auth_method: /governance/auth/sso-entra-oidc
+        sub_claims: { groups: [grp-fraud-analysts] }
```

New expectations (one positive, two negative), the expected plan (+1 role, +1 association) and the approvers from CODEOWNERS: payments owners and the platform team.

8.4 `review`

Syntax
`/akeyless-watchover review <model files, merge-request diff or plan output>`

Arguments
<model files, merge-request diff or plan output>

Purpose
Review a proposed change for least privilege, guard violations and blast radius.

Delivers
Findings by severity, failing guards, missing expectations, plan red flags (replacements).

Example
`/akeyless-watchover review <paste the plan>`

Example session

You /akeyless-watchover review *[pasted plan]*
Claude **High**: `akeyless_auth_method_k8s.this["/governance/auth/k8s-prod"]` *must be replaced* – the name changed; consumers lose their access ID. Revert the rename or plan a cut-over. **Medium**: rule `/payments/*` with `update` on `team-admin` widens access to every app's prod; no expectation covers it. **OK**: 2 in-place rule updates match the request.

8.5 `pipeline`

Syntax
`/akeyless-watchover pipeline [github | gitlab] [repository]`

Arguments
[github | gitlab] [repository]

Purpose
Set up the governance repository: bootstrap identity, CI, CODEOWNERS, state.

Delivers
bootstrap.tfvars, CI file, variables to create, step-by-step order.

Example

```
/akeyless-watchover pipeline gitlab group/akeyless-governance
```

Example session

You /akeyless-watchover pipeline gitlab acme/platform/akeyless-governance
Claude `bootstrap.tfvars` (`platform = "gitlab"`, `repository = "acme/platform/akeyless-governance"`, `gitlab_url`), the order of steps (bootstrap as admin → CI variables → copy `ci/gitlab/.gitlab-ci.yml` → protected branch and environment → CODEOWNERS), the list of CI variables to create and which ones must be masked.

8.6 `import`

Syntax

```
/akeyless-watchover import [what to export: everything / a type / one user's
view] [export directory or REPORT.md]
```

Arguments

[what to export: everything / a type / one user's view] [export directory or REPORT.md]

Purpose

Export existing config (RBAC, auth methods incl. OIDC, folders, custom fields, key policies, PKI issuers, account settings) and translate it into Terraform.

Delivers

The exact `watchover export` command (model or --hcl mode, filters), then resolution of every REPORT.md item and the import checklist until the plan shows imports only.

Example

```
/akeyless-watchover import the OIDC auth method and everything alice@example.
com can reach
```

Example session

You /akeyless-watchover import the OIDC auth method and everything alice@example.com can reach
Claude The command to run:

```
scripts/watchover export --out export/ --profile admin --types roles,auth-methods \
  --persona-auth-method /governance/auth/sso-entra-oidc \
  --persona-claim email=alice@example.com --persona-claim groups=grp-akl-payments-admins
```

Then, given `export/REPORT.md`: one decision per report item (client secret into `TF_VAR_auth_method_secrets`, a temporary rule that will not be imported, an association to an unmanaged auth method), where to move each file, and "iterate until the plan shows imports only".

8.7 `conjur`

Syntax

```
/akeyless-watchover conjur <Conjur policy YAML>
```

Arguments

<Conjur policy YAML>

Purpose
Migrate from CyberArk Conjur: secrets via the Gateway, RBAC via translation.

Delivers
Translated roles, bindings to fill (auth method + sub-claims), migration resource, cut-over plan.

Example
`/akeyless-watchover conjur <attach policy files>`

Example session

You /akeyless-watchover conjur *[attached root.yml and prod.yml]*
Claude Runs (or tells you to run) `scripts/watchover conjur root.yml prod.yml --out export/conjur`, explains each translated role (Conjur layer/group → Akeyless role on the migrated paths), and for every entry of `bindings.yaml` proposes the auth method and sub-claims (e.g. host `prod/payments/checkout` → K8s auth, namespace `payments`). Plus the Gateway migration entry for `model/gateway.yaml` and a cut-over plan.

8.8 `pki`

Syntax
`/akeyless-watchover pki <certificate requirements>`

Arguments
<certificate requirements>

Purpose
Design private/public CA issuers and who may request certificates.

Delivers
pki.yaml (CA keys, issuers), requester roles, expiry forwarder, CLI to issue/revoke.

Example
`/akeyless-watchover pki internal TLS for *.payments.internal, 90 days, auto-renew`

Example session

You /akeyless-watchover pki internal TLS for *.payments.internal, 90 days, auto-renew
Claude `model/pki.yaml` with a CA key (`/pki/ca/internal-root`) and an issuer (`/pki/issuers/payments-server`, `allowed_domains`, TTL 7776000 s, `auto_renew` with `destination_path`); a requester role for the payments workloads; an expiry forwarder (`watchover forwarder --preset certificates`); the CLI to issue and revoke a certificate.

8.9 `harden`

Syntax
`/akeyless-watchover harden [current settings or concerns]`

Arguments
[current settings or concerns]

Purpose
Account-level hardening.

Delivers
governance.yaml: account_settings (Terraform), account_settings_cli (CLI), custom fields, key policies, event forwarders.

Example
`/akeyless-watchover harden`

Example session

You /akeyless-watchover harden
Claude Proposals with reasons, split by where they are delivered: `account_settings` (Terraform: password policy, max versions, JWT TTLs, item deletion protection), `account_settings_cli` (bound IPs, allowed client types, locks – applied by `watchover account-settings`; *dry-run first*), mandatory custom fields such as `owner`, key policies, and critical event forwarders.

8.10 `gateway`

Syntax
`/akeyless-watchover gateway <who administers which Gateway, SIEM target>`

Arguments
<who administers which Gateway, SIEM target>

Purpose
Gateway admin plane and audit-log forwarding.

Delivers
gateway.yaml (allowed_access, log_forwarding), gateway_api setup.

Example
`/akeyless-watchover gateway platform admins full, PKI team targets only, Splunk`

Example session

You /akeyless-watchover gateway platform admins full, PKI team targets only, Splunk
Claude `model/gateway.yaml`: `allowed_access` entries (platform admins through the SSO auth method with their group sub-claim; PKI team limited to the targets permission), `log_forwarding` of type `splunk` with `splunk_token_var`, and the reminder to set `gateway_api` – one state per Gateway cluster.

8.11 `verify`

Syntax
`/akeyless-watchover verify [persona | path]`

Arguments
[persona | path]

Purpose
Prove who can access what.

Delivers
access_expectations.yaml checks, `watchover simulate` / `watchover verify` commands, Impersonate As steps.

Example
`/akeyless-watchover verify can the checkout pods read refunds prod?`

Example session

You /akeyless-watchover verify can the checkout pods read refunds prod?
Claude **No.** `checkout-prod-workload` only matches `/payments/checkout/prod/*`; no other role is associated with the K8s auth method. Adds the negative check `{ persona: checkout-prod-pod, path: /payments/refunds/prod/db, expect: [] }` and shows how to prove it offline (`watchover simulate --explain`) and live (`watchover verify`, or *Impersonate As* in the console).

8.12 `cli`

Syntax
`/akeyless-watchover cli <task>`

Arguments
<task>

Purpose
Exact Akeyless CLI commands for a task (break-glass, bootstrap, one-off checks).

Delivers
Commands checked against the CLI reference, with the equivalent model change.

Example
`/akeyless-watchover cli create a temporary read rule for 60 minutes`

Example session

You /akeyless-watchover cli create a temporary read rule for 60 minutes
Claude

```
akeyless set-role-rule --role-name /governance/roles/payments/team-admin \
  --path "/payments/refunds/prod/*" --capability read --capability list --ttl 60
```

With the caveat that temporary rules are CLI/API-only and the next apply on that role removes them – and the model change to make if the access should last.

8.13 `explain`

Syntax
`/akeyless-watchover explain <topic>`

Arguments
<topic>

Purpose

Explain an Akeyless concept precisely (sub-claims, deny, wildcards, anti-escalation…).

Delivers

A short answer with the verified facts and VERIFY caveats.

Example

```
/akeyless-watchover explain should this run through the API or the CLI?
```

Example session

You /akeyless-watchover explain how do sub-claims combine?

Claude Within one key the listed values are alternatives (OR); across keys all must match (AND). With `force_sub_claims` on the auth method, an association without sub-claims is rejected – the kit's guard enforces it before Akeyless does. Glob patterns in values are supported; case sensitivity is per association.

Chapter 9

Kit commands (`scripts/watchover`)

The kit runs in your governance repository: `scripts/watchover <command> [options]`. `scripts/watchover help` prints the same information in the terminal and `scripts/watchover <command> --help` shows one command. Each section lists *all* options: **required** options must be given, **repeatable** options may be given several times, and a default is shown when the option has one. Each section ends with a real execution, captured when this manual was built (see Section 1.5.1).

Command	Summary	Akeyless
`test`	Run every offline check (what CI runs).	offline
`simulate`	Offline: evaluate access_expectations.yaml against the model.	offline
`verify`	Live: compare expectations with describe-permissions run as each persona's CLI profile.	live
`export`	Export existing config and translate it: YAML model + import blocks, or plain Terraform (--hcl).	live
`forwarder`	Scaffold a new event forwarder (model entry or HCL); --import-existing adopts one by name.	offline
`conjur`	Translate Conjur policy into model roles + identity bindings.	offline
`account-settings`	Apply account_settings_cli (settings the provider cannot manage) with the CLI.	live
`token`	CI: exchange the pipeline JWT for an Akeyless token.	live
`build-skill`	Package the skill ZIP.	offline
`help`	Help in the terminal; `--markdown` regenerates docs/usage-guide.md.	offline
`version`	Print the version.	offline

9.1 `watchover test`

Run every offline check (what CI runs). *Offline.*

```
Run the complete offline test suite (no Akeyless account needed): fmt, validate, terraform test,
↪  guard messages, export round trip, bootstrap tests, access simulation, script tests, guide
↪  drift check.
```

Options.

Option	Value	Required / default	Description
`--tool`	`terraform|tofu`	default terraform	IaC binary to use
`--skip-iac`	–	optional	only run the Python checks

Usage patterns.

```
scripts/watchover test
scripts/watchover test --tool tofu
```

Example execution

Every offline check, as CI runs them (formatting, validation, Terraform tests with a mocked provider, guard messages, export round trip, bootstrap tests, Python tests, access simulation):

```
$ scripts/watchover test --tool tofu

$ tofu fmt -check -recursive   (in terraform)

$ tofu init -backend=false -input=false   (in terraform)

Initializing provider plugins...
- terraform.io/builtin/terraform is built in to OpenTofu
- Reusing previous version of akeyless-community/akeyless from the dependency lock file
- Installing akeyless-community/akeyless v2.0.5...
- Installed akeyless-community/akeyless v2.0.5 (unauthenticated)

OpenTofu has been successfully initialized!

$ tofu validate   (in terraform)
Success! The configuration is valid.

$ tofu test   (in terraform)
tests/model.tftest.hcl... pass
  run "reference_model_plans"... pass
  run "gateway_objects_need_gateway_api"... pass
  run "ara_rule_allowed_when_enabled"... pass
  run "root_scope_allowed_with_flag"... pass
  run "dup"... pass
  run "deny_mixed"... pass
  run "bad_capability"... pass
  run "bad_rule_type"... pass
  run "sra_conflict"... pass
  run "root_scope"... pass
  run "unknown_am"... pass
  run "missing_subclaims"... pass
  run "bad_prefix"... pass
  run "bad_mgmt"... pass
  run "bad_issuer"... pass
  run "bad_gateway"... pass
  run "bad_event"... pass
  run "scaffolded_forwarders_plan"... pass
  run "bad_periodic"... pass
  run "slack_no_url"... pass
  run "empty_gateway_api_is_unset"... pass
  run "naming_check_disabled_with_empty_string"... pass
[... 37 lines omitted ...]
Success! The configuration is valid.

$ tofu test   (in bootstrap)
tests/bootstrap.tftest.hcl... pass
  run "github"... pass
  run "gitlab"... pass
  run "bad_platform"... pass

Success! 3 passed, 0 failed.

$ /usr/local/bin/python3 ./scripts/akl_simulate.py --model terraform/model --expectations
↪  terraform/model/access_expectations.yaml   (in .)
PASS  payments-admin           item        /payments/checkout/nonprod/db/app
↪  expect=['create', 'delete', 'list', 'read', 'update'] got=['create', 'delete', 'list', 'read', 'update']
PASS  payments-admin           item        /payments/checkout/prod/db/app
↪  expect=['list', 'read'] got=['list', 'read']
PASS  payments-admin           item        /payments/checkout/prod/break-glass/root           expect=[]
↪  got=[]
PASS  payments-admin           item        /hr/payroll/prod/db                                expect=[]
↪  got=[]
PASS  auditor                  item        /payments/checkout/prod/db/app
↪  expect=['list'] got=['list']
PASS  auditor                  role        /governance/roles/payments/team-admin
↪  expect=['list', 'read'] got=['list', 'read']
```

```
PASS  random-employee          item        /payments/checkout/prod/db/app                    expect=[]
↪  got=[]
PASS  checkout-prod-pod        item        /payments/checkout/prod/db/app
↪  expect=['list', 'read'] got=['list', 'read']
PASS  checkout-prod-pod        item        /payments/checkout/prod/break-glass/root          expect=[]
↪  got=[]
PASS  checkout-prod-pod        item        /payments/checkout/nonprod/db/app                 expect=[]
↪  got=[]
PASS  checkout-prod-pod        item        /pki/issuers/payments-server
↪  expect=['list', 'read'] got=['list', 'read']

0 mismatch(es)

$ /usr/local/bin/python3 -m pytest -q scripts/tests   (in .)
.....................................................................                    [100%]
71 passed in <t>s

all checks passed
```

exit code 0

Only the Python checks:

```
$ scripts/watchover test --skip-iac

$ /usr/local/bin/python3 ./scripts/akl_simulate.py --model terraform/model --expectations
↪  terraform/model/access_expectations.yaml   (in .)
PASS  payments-admin           item        /payments/checkout/nonprod/db/app
↪  expect=['create', 'delete', 'list', 'read', 'update'] got=['create', 'delete', 'list', 'read', 'update']
PASS  payments-admin           item        /payments/checkout/prod/db/app
↪  expect=['list', 'read'] got=['list', 'read']
PASS  payments-admin           item        /payments/checkout/prod/break-glass/root          expect=[]
↪  got=[]
PASS  payments-admin           item        /hr/payroll/prod/db                               expect=[]
↪  got=[]
PASS  auditor                  item        /payments/checkout/prod/db/app
↪  expect=['list'] got=['list']
PASS  auditor                  role        /governance/roles/payments/team-admin
↪  expect=['list', 'read'] got=['list', 'read']
PASS  random-employee          item        /payments/checkout/prod/db/app                    expect=[]
↪  got=[]
PASS  checkout-prod-pod        item        /payments/checkout/prod/db/app
↪  expect=['list', 'read'] got=['list', 'read']
PASS  checkout-prod-pod        item        /payments/checkout/prod/break-glass/root          expect=[]
↪  got=[]
PASS  checkout-prod-pod        item        /payments/checkout/nonprod/db/app                 expect=[]
↪  got=[]
PASS  checkout-prod-pod        item        /pki/issuers/payments-server
↪  expect=['list', 'read'] got=['list', 'read']

0 mismatch(es)

$ /usr/local/bin/python3 -m pytest -q scripts/tests   (in .)
.....................................................................                    [100%]
71 passed in <t>s

all checks passed
```

exit code 0

9.2 `watchover simulate`

Offline: evaluate access_expectations.yaml against the model. *Offline.*

```
Offline access check of the YAML model against expected access (no Akeyless calls).

Usage:
```

```
  python3 scripts/akl_simulate.py --model terraform/model       --expectations
  ↪  terraform/model/access_expectations.yaml [--explain]

Exit code 0 = all expectations met, 1 = mismatch, 2 = usage/model error.
```

Options.

Option	Value	Required / default	Description
`--model`	`MODEL`	**required**	model directory (terraform/model)
`--expectations`	`EXPECTATIONS`	**required**	access_expectations.yaml
`--explain`	—	optional	print the matching rules for every check, not only failures

Usage patterns.

```
scripts/watchover simulate --model terraform/model --expectations
↪  terraform/model/access_expectations.yaml --explain
```

Example execution

Offline evaluation of the access expectations against the model:

```
$ scripts/watchover simulate --model terraform/model --expectations terraform/model/access_expectations.yaml
PASS  payments-admin           item        /payments/checkout/nonprod/db/app
↪  expect=['create', 'delete', 'list', 'read', 'update'] got=['create', 'delete', 'list', 'read', 'update']
PASS  payments-admin           item        /payments/checkout/prod/db/app
↪  expect=['list', 'read'] got=['list', 'read']
PASS  payments-admin           item        /payments/checkout/prod/break-glass/root           expect=[]
↪  got=[]
PASS  payments-admin           item        /hr/payroll/prod/db                                expect=[]
↪  got=[]
PASS  auditor                  item        /payments/checkout/prod/db/app
↪  expect=['list'] got=['list']
PASS  auditor                  role        /governance/roles/payments/team-admin
↪  expect=['list', 'read'] got=['list', 'read']
PASS  random-employee          item        /payments/checkout/prod/db/app                     expect=[]
↪  got=[]
PASS  checkout-prod-pod        item        /payments/checkout/prod/db/app
↪  expect=['list', 'read'] got=['list', 'read']
PASS  checkout-prod-pod        item        /payments/checkout/prod/break-glass/root           expect=[]
↪  got=[]
PASS  checkout-prod-pod        item        /payments/checkout/nonprod/db/app                  expect=[]
↪  got=[]
PASS  checkout-prod-pod        item        /pki/issuers/payments-server
↪  expect=['list', 'read'] got=['list', 'read']

0 mismatch(es)
```

exit code 0

With `--explain`, every check shows the rules that produced the result:

```
$ scripts/watchover simulate --model terraform/model --expectations terraform/model/access_expectations.yaml
↪  --explain | head -n 12
PASS  payments-admin           item        /payments/checkout/nonprod/db/app
↪  expect=['create', 'delete', 'list', 'read', 'update'] got=['create', 'delete', 'list', 'read', 'update']
       /governance/roles/payments/team-admin: /payments/+/nonprod/* -> ['create', 'delete', 'list', 'read',
       ↪  'update']
PASS  payments-admin           item        /payments/checkout/prod/db/app
↪  expect=['list', 'read'] got=['list', 'read']
       /governance/roles/payments/team-admin: /payments/+/prod/* -> ['list', 'read']
```

```
PASS  payments-admin          item       /payments/checkout/prod/break-glass/root          expect=[]
↪  got=[]
       /governance/roles/payments/team-admin: /payments/+/prod/* -> ['list', 'read']
       /governance/roles/payments/team-admin: /payments/+/prod/break-glass/* -> ['deny']
       deny overrides all other permissions
PASS  payments-admin          item       /hr/payroll/prod/db                               expect=[]
↪  got=[]
       no matching rule
PASS  auditor                 item       /payments/checkout/prod/db/app
↪  expect=['list'] got=['list']
       /governance/roles/platform/auditors: /* -> ['list']
```

exit code 0

9.3 `watchover verify`

Live: compare expectations with describe-permissions run as each persona's CLI profile. *Calls Akeyless.*

```
Live access verification against Akeyless (after apply).

For every check in the expectations file, runs - as the persona's CLI profile -
    akeyless describe-permissions --path <path> --type <item|am|role|target> --json [--profile P]
which returns the *caller's* own permissions on that path ({"client_permissions": [...]}),
and compares them with the expected set.

Personas need a `profile:` (an `akeyless configure` profile authenticating as that
persona, e.g. a test SSO user in the right IdP group or a test workload identity).
Personas without a profile are skipped (use the Console "Impersonate As" for those).

Usage:
  python3 scripts/akl_verify.py --expectations terraform/model/access_expectations.yaml [--akeyless
  ↪  akeyless]
Exit code 0 = all checked expectations met, 1 = mismatch, 2 = execution error.
```

Options.

Option	Value	Required / default	Description
`--expectations`	`EXPECTATIONS`	**required**	access_expectations.yaml (personas need a 'profile')
`--akeyless`	`AKEYLESS`	default akeyless	path to the akeyless CLI binary
`--error-means-no-access`	—	optional	treat a failing describe-permissions call as 'no permissions' (check the CLI behaviour first)

Usage patterns.

```
scripts/watchover verify --expectations terraform/model/access_expectations.yaml
```

Example execution

Live check: every persona has a CLI profile (`profile:` in access_expectations.yaml). The demo account contains one change made in the UI and not in Git - a role that grants everybody read on checkout prod - and verify catches it (exit code 1):

```
$ scripts/watchover verify --expectations terraform/model/access_expectations.yaml
PASS  payments-admin          /payments/checkout/nonprod/db/app                expect=['create',
↪  'delete', 'list', 'read', 'update'] got=['create', 'delete', 'list', 'read', 'update']
PASS  payments-admin          /payments/checkout/prod/db/app                   expect=['list', 'read']
↪  got=['list', 'read']
PASS  payments-admin          /payments/checkout/prod/break-glass/root         expect=[] got=[]
PASS  payments-admin          /hr/payroll/prod/db                              expect=[] got=[]
PASS  auditor                 /payments/checkout/prod/db/app                   expect=['list']
↪  got=['list']
PASS  auditor                 /governance/roles/payments/team-admin            expect=['list', 'read']
↪  got=['list', 'read']
FAIL  random-employee         /payments/checkout/prod/db/app                   expect=[] got=['list',
↪  'read']
PASS  checkout-prod-pod       /payments/checkout/prod/db/app                   expect=['list', 'read']
↪  got=['list', 'read']
PASS  checkout-prod-pod       /payments/checkout/prod/break-glass/root         expect=[] got=[]
PASS  checkout-prod-pod       /payments/checkout/nonprod/db/app                expect=[] got=[]
PASS  checkout-prod-pod       /pki/issuers/payments-server                     expect=['list', 'read']
↪  got=['list', 'read']

1 mismatch(es), 0 check(s) skipped (persona without profile)
```

exit code 1

Explicit CLI binary; a failing describe-permissions call would count as 'no access' (last lines):

```
$ scripts/watchover verify --expectations terraform/model/access_expectations.yaml --akeyless akeyless
↪  --error-means-no-access
[... 10 lines omitted ...]
PASS  checkout-prod-pod       /pki/issuers/payments-server                     expect=['list', 'read']
↪  got=['list', 'read']

1 mismatch(es), 0 check(s) skipped (persona without profile)
```

exit code 1

9.4 `watchover export`

Export existing config and translate it: YAML model + import blocks, or plain Terraform (--hcl). *Calls Akeyless.*

```
Export an existing Akeyless account and translate it into Terraform.

Two output modes (both read the same objects with the akeyless CLI, or from saved JSON):

  model (default)  YAML model for terraform/ + import blocks for its addresses
                   <out>/model/roles/imported.yaml          roles, rules, management plane,
                   ↪  associations
                   <out>/model/auth_methods.imported.yaml   auth methods incl. type-specific
                   ↪  settings
                   <out>/model/folders.imported.yaml        explicit folders (description, tags,
                   ↪  protection)
                   <out>/model/governance.imported.yaml     custom fields, key policies
                   <out>/imports.tf                         import {} blocks for terraform/*.tf
  hcl (--hcl)      Plain Terraform generated by Terraform itself from the live objects
                   <out>/hcl/providers.tf + imports.tf      flat import blocks for EVERY importable
                   ↪  object found
                                                            (also PKI issuers and account settings)
                   <out>/hcl/generated.tf                   with --generate: `terraform plan
                   ↪  -generate-config-out`

Always written: <out>/raw/ (every JSON read, secrets redacted - reusable with --from-dir)
and <out>/REPORT.md (everything that needs a human decision).
```

```
Selection: --types limits object families; --only keeps names matching a regex;
--persona-auth-method + --persona-claim keep one identity's view (its auth method and the roles
whose association sub-claims match it - e.g. "the OIDC config and access of one user").

Usage:
  watchover export --out export/ --profile admin
  watchover export --out export/ --profile admin --types auth-methods --only 'oidc'
  watchover export --out export/ --profile admin --persona-auth-method
  ↪  /governance/auth/sso-entra-oidc \
      --persona-claim email=alice@example.com --persona-claim groups=grp-akl-payments-admins
  watchover export --out export/ --profile admin --hcl --generate      # needs terraform +
  ↪  AKEYLESS_ACCESS_ID/KEY
  watchover export --out export/ --from-dir export-old/raw              # offline replay
```

Options.

Option	Value	Required / default	Description
`--out`	`OUT`	**required**	output directory (model/, imports.tf or hcl/, raw/, REPORT.md)
`--transport`	`cli|api`	default cli	how to read Akeyless: the akeyless CLI (profiles, SSO logins) or the REST API directly (token)
`--profile`	`PROFILE`	optional	cli transport: akeyless CLI profile with read access to the objects to export
`--akeyless`	`AKEYLESS`	default akeyless	cli transport: path to the akeyless CLI binary
`--gateway-url`	`GATEWAY_URL`	optional	cli transport, --types gateway: Gateway configuration URL (https://gw:8000)
`--api`	`API`	default https://api.akeyless.io	api transport: API base URL (default $AKEYLESS_API or https://api.akeyless.io)
`--token`	`TOKEN`	optional	api transport: Akeyless token (default $AKEYLESS_TOKEN; e.g. from 'watchover token')
`--gateway-api`	`GATEWAY_API`	optional	api transport, --types gateway: Gateway V2 API (https://gw:8000/api/v2)
`--from-dir`	`FROM_DIR`	optional	replay a raw/ directory of a previous export instead of calling the CLI
`--types`	`TYPES`	default roles,auth-methods,folders,custom-fields,key-policies,pki-issuers,account-settings,targets,groups	comma-separated object families (default: all but gateway): roles, auth-methods, folders, custom-fields, key-policies, pki-issuers, account-settings, targets, groups, gateway
`--only`	`ONLY`	optional	keep only objects whose name matches this regex
`--exclude`	`EXCLUDE`	*repeatable*	regex of names to skip (repeatable)
`--root`	`ROOT`	default /	folder to start the item/folder walk from

Option	Value	Required / default	Description
`--max-depth`	`MAX_DEPTH`	default 20	maximum folder depth of the walk
`--persona-auth-method`	`PERSONA_AUTH_METHOD`	optional	keep one identity's view: its auth method ...
`--persona-claim`	`PERSONA_CLAIM`	*repeatable*	... and its claims, key=value (repeatable); keeps roles whose association matches
`--hcl`	–	optional	write plain Terraform import blocks for every object (hcl/)
`--generate`	–	optional	with --hcl: run 'terraform plan -generate-config-out' (needs terraform and AKEYLESS_ACCESS_ID/AKEYLESS_ACCESS_KEY in the environment)
`--terraform`	`TERRAFORM`	default terraform	terraform or tofu binary used by --generate

Usage patterns.

```
scripts/watchover export --out export/ --profile admin --exclude '^/governance/auth/ci-terraform$'
scripts/watchover export --out export/ --profile admin --types auth-methods --only sso
scripts/watchover export --out export/ --profile admin --persona-auth-method
↪  /governance/auth/sso-entra-oidc --persona-claim email=alice@example.com --persona-claim
↪  groups=grp-akl-payments-admins
AKEYLESS_ACCESS_ID=p-... AKEYLESS_ACCESS_KEY=... scripts/watchover export --out export/ --profile
↪  admin --hcl --generate
AKEYLESS_TOKEN=$(scripts/watchover token) scripts/watchover export --out export/ --transport api
↪  --hcl --types targets,groups,gateway --gateway-api https://gw.example.com:8000/api/v2
scripts/watchover export --out export2/ --from-dir export/raw     # offline replay
```

Example execution

Model mode through the CLI: YAML model files, import blocks, a review report and the raw JSON:

```
$ scripts/watchover export --out export/ --profile admin --exclude '^/governance/auth/ci-terraform$'
exported (model): 3 roles, 6 auth-methods, 1 folders, 1 custom-fields, 1 key-policies, 1 pki-issuers, 1
↪  account-settings, 3 targets, 1 groups; 14 item(s) in REPORT.md
```

exit code 0

What was written:

```
$ find export -maxdepth 2 | sort
export
export/REPORT.md
export/imports.tf
export/model
export/model/auth_methods.imported.yaml
export/model/folders.imported.yaml
export/model/governance.imported.yaml
export/model/roles
export/raw
export/raw/account_settings.json
export/raw/auth_methods
export/raw/custom_fields.json
export/raw/folders
export/raw/folders_discovered.json
export/raw/groups.json
export/raw/items
export/raw/policies.json
```

```
export/raw/roles
export/raw/targets.json
```

exit code 0

The review report (first lines):

```
$ sed -n '1,30p' export/REPORT.md
# Export report

mode: model; roles: 3, auth-methods: 6, folders: 1, custom-fields: 1, key-policies: 1, pki-issuers: 1,
↪  account-settings: 1, targets: 3, groups: 1

- folders: 5 discovered, 1 explicit folder object(s) exported (implicit folders exist through item paths and
↪  need no resource)
- items walked: 2 (item types: PKI_CERT_ISSUER, STATIC_SECRET); secrets, keys and targets are not exported -
↪  only PKI issuers
- role `/governance/roles/platform/god`: root-scope rule `item-rule /* ['read', 'list', 'create', 'update',
↪  'delete']` - confirm it is intended
- role `/governance/roles/payments/team-admin`: temporary rule on `/payments/tmp` (ttl) skipped - temporary
↪  rules are CLI-only
- role `/governance/roles/payments/team-admin`: rule type `brand-new-rule` on `/x` is not handled by the
↪  model - review
- auth method `/governance/auth/k8s-prod` (k8s): also needs the Gateway-side K8s auth config (out of scope)
- auth method `/legacy/kerb`: type `kerberos_access_rules` is not modelled - manage it outside the model, or
↪  use --hcl to get plain Terraform for it
- auth method `/legacy/apikey`: API key - after import its access key will be in Terraform state
- auth method `/governance/auth/sso`: client secret not exported - put it in TF_VAR_auth_method_secrets
↪  under `governance_auth_sso_client_secret`
- role `/governance/roles/payments/team-admin` is associated with `/governance/auth/ci-terraform`, which is
↪  not in this export
- 1 PKI issuer(s) found: model mode does not translate issuers (nested API format). Run again with --hcl
↪  --types pki-issuers for plain Terraform, or write pki.yaml from raw/items/
- account settings exported to raw/account_settings.json; use --hcl --types account-settings for the
↪  Terraform-managed subset, and account_settings_cli for the rest
- 3 target(s): not part of the YAML model - use --hcl --types targets for plain Terraform (raw JSON in raw/)
- 1 group(s): not part of the YAML model - use --hcl --types groups for plain Terraform (raw JSON in raw/)
```

exit code 0

The translated roles (one file per top-level folder; first lines of the first one):

```
$ ls export/model/roles && sed -n '1,24p' "$(ls -d export/model/roles/* | head -n 1)"
imported.yaml
roles:
  /governance/roles/payments/team-admin:
    allow_external_auth_methods: true
    analytics_access: all
    associations:
    - auth_method: /governance/auth/sso
      case_sensitive: false
      sub_claims:
        groups:
        - grp-a
        - grp-b
    - auth_method: /governance/auth/ci-terraform
    audit_access: scoped
    delete_protection: true
    description: Payments engineers
    rules:
    - capabilities:
      - create
      - list
      - read
      path: /payments/+/nonprod/*
    - capabilities:
      - deny
      path: /payments/+/prod/break-glass/*
```

exit code 0

Plain-Terraform mode (`--hcl`): flat import blocks; add `--generate` and Terraform writes the HCL itself with `plan -generate-config-out` (needs live credentials, so not executed here):

```
$ scripts/watchover export --out export-hcl/ --profile admin --hcl --types roles,auth-methods && sed -n
↪  '1,12p' export-hcl/hcl/imports.tf
exported (hcl): 3 roles, 7 auth-methods; 6 item(s) in REPORT.md
# Generated by watchover export --hcl. Run:
#   terraform init && terraform plan -generate-config-out=generated.tf
# Terraform reads every object and writes its resource block into generated.tf.

import {
  to = akeyless_auth_method_aws_iam.governance_auth_aws_payments_prod
  id = "/governance/auth/aws-payments-prod"
}

import {
  to = akeyless_auth_method_azure_ad.governance_auth_azure_prod
  id = "/governance/auth/azure-prod"
```

exit code 0

One user's view (the persona filter): the auth method alice signs in with and only the roles her claims reach:

```
$ scripts/watchover export --out export-persona/ --profile admin --types roles,auth-methods
↪  --persona-auth-method /governance/auth/sso --persona-claim email=alice@example.com --persona-claim
↪  groups=grp-a && grep -h '^  /' export-persona/model/*.yaml export-persona/model/roles/*.yaml
exported (model): 1 roles, 1 auth-methods; 6 item(s) in REPORT.md
  /governance/auth/sso:
  /governance/roles/payments/team-admin:
```

exit code 0

REST transport (no CLI binary), including the Gateway families: (Shown as you would run it; executed against the local stand-in endpoint.)

```
$ AKEYLESS_TOKEN=$(scripts/watchover token) scripts/watchover export --out export-api/ --transport api --api
↪  https://api.akeyless.io --types targets,groups,gateway --gateway-api https://gw.example.com:8000/api/v2
↪  --hcl && sed -n '1,14p' export-api/hcl/imports.tf
exported (hcl): 3 targets, 1 groups, 1 gateway; 7 item(s) in REPORT.md
# Generated by watchover export --hcl. Run:
#   terraform init && terraform plan -generate-config-out=generated.tf
# Terraform reads every object and writes its resource block into generated.tf.

import {
  to = akeyless_target_aws.targets_aws_prod
  id = "/targets/aws-prod"
}

import {
  to = akeyless_target_lets_encrypt.targets_letsencrypt
  id = "/targets/letsencrypt"
}
```

exit code 0

Offline replay of a previous export (no Akeyless access needed):

```
$ scripts/watchover export --out export-replay/ --from-dir export/raw
exported (model): 3 roles, 7 auth-methods, 1 folders, 1 custom-fields, 1 key-policies, 1 pki-issuers, 1
↪  account-settings, 3 targets, 1 groups; 13 item(s) in REPORT.md
```

exit code 0

9.5 `watchover forwarder`

Scaffold a new event forwarder (model entry or HCL); --import-existing adopts one by name. *Offline.*

```
Scaffold Terraform configuration for Akeyless event forwarders.

Akeyless has no operation that lists event forwarders, so they can't be exported. This command
writes new ones instead, already validated against the same rules the Terraform guards enforce:

  * model entry for terraform/model/governance.yaml (default; --write inserts it in place)
  * plain HCL resource block (--format hcl) for repositories that don't use the YAML model
  * an import block (--import-existing) to bring an EXISTING forwarder whose name you know under
    Terraform: the provider imports event forwarders by name

Secrets (webhook passwords/tokens/certificates, Slack/Teams URLs, ServiceNow credentials) are
never written: the output references a *_var key you then provide in
↪  TF_VAR_event_forwarder_secrets.

Examples:
  watchover forwarder --name /governance/events/secops-mail --type email --email-to
  ↪  secops@example.com \
      --preset critical --items-source '/*'
  watchover forwarder --name /governance/events/pki-slack --type slack --preset certificates \
      --items-source '/pki/*' --write terraform/model/governance.yaml
  watchover forwarder --name /governance/events/siem --type webhook --url
  ↪  https://siem.example.com/akeyless \
      --auth-type bearer-token --preset critical --auth-method-source '/*' --format hcl
  watchover forwarder --name /governance/events/old-mail --type email --email-to a@b.c --event
  ↪  certificate-expired \
      --import-existing
```

Options.

Option	Value	Required / default	Description
`--name`	`NAME`	**required**	forwarder name, e.g. /governance/events/secops-mail
`--type`	`email|webhook|slack|teams|service_now`	**required**	forwarder type
`--description`	`DESCRIPTION`	optional	description
`--preset`	`access-requests|auth-methods|certificates|critical|operations|rotation|secrets`	*repeatable*	add a group of event types (repeatable); see 'Presets' in the usage guide
`--event`	`EVENT`	*repeatable*	add one event type (repeatable)
`--items-source`	`ITEMS_SOURCE`	*repeatable*	item path pattern to watch, e.g. '/pki/*' (repeatable)
`--targets-source`	`TARGETS_SOURCE`	*repeatable*	target path pattern to watch (repeatable)
`--auth-method-source`	`AUTH_METHOD_SOURCE`	*repeatable*	auth method path pattern to watch (repeatable)

Option	Value	Required / default	Description
`--gateway-source`	`GATEWAY_SOURCE`	*repeatable*	Gateway cluster URL to watch (repeatable; required for teams)
`--runner`	`immediate|periodic`	default immediate	send at once or as a periodic digest
`--every`	`EVERY`	optional	hours between digests (required with --runner periodic)
`--disabled`	—	optional	create it disabled
`--key`	`KEY`	optional	protection key for the forwarder's secret (default: account key)
`--email-to`	`EMAIL_TO`	*repeatable*	email: recipient (repeatable)
`--include-error`	—	optional	email: include error details
`--override-url`	`OVERRIDE_URL`	optional	email: Gateway URL used in links (port 18888)
`--url`	`URL`	optional	webhook: URL (slack/teams URLs are secrets and go in the secrets map)
`--auth-type`	`AUTH_TYPE`	optional	webhook: user-pass/bearer-token/certificate; service_now: user-pass/jwt
`--username`	`USERNAME`	optional	webhook: user for user-pass
`--host`	`HOST`	optional	service_now: instance host
`--admin-name`	`ADMIN_NAME`	optional	service_now: admin name (user-pass)
`--client-id`	`CLIENT_ID`	optional	service_now: client ID (jwt)
`--user-email`	`USER_EMAIL`	optional	service_now: user email (jwt)
`--format`	`yaml|hcl`	default yaml	model entry (yaml) or plain resource (hcl)
`--write`	`GOVERNANCE_YAML`	optional	yaml: insert the entry into this governance.yaml
`--import-existing`	—	optional	the forwarder already exists in Akeyless: also print the import block (import ID = name)

Usage patterns.

```
scripts/watchover forwarder --name /governance/events/secops-mail --type email --email-to
↪  secops@example.com --preset critical --items-source '/*' --auth-method-source '/*'
scripts/watchover forwarder --name /governance/events/pki-slack --type slack --preset certificates
↪  --items-source '/pki/*' --write terraform/model/governance.yaml
scripts/watchover forwarder --name /governance/events/siem --type webhook --url
↪  https://siem.example.com/akeyless --auth-type bearer-token --preset critical
↪  --auth-method-source '/*' --format hcl
scripts/watchover forwarder --name /governance/events/legacy-mail --type email --email-to
↪  ops@example.com --event certificate-expired --items-source '/*' --import-existing
```

Example execution

A model entry (default output):

```
$ scripts/watchover forwarder --name /governance/events/secops-mail --type email --email-to
↪  secops@example.com --preset critical --items-source '/*' --auth-method-source '/*'
# add under event_forwarders: in terraform/model/governance.yaml
  /governance/events/secops-mail:
    type: email
    event_types:
    - certificate-expired
    - certificate-provisioning-failure
```

```
    - auth-method-expired
    - rotated-secret-failure
    - dynamic-secret-failure
    - multi-auth-failure
    - uid-rotation-failure
    - gateway-inactive
    runner_type: immediate
    items_event_source_locations:
    - /*
    auth_methods_event_source_locations:
    - /*
    email_to:
    - secops@example.com
```

exit code 0

Insert it into the model (comments and the rest of the file are kept; the secret key is printed): (The sandbox copy of governance.yaml is restored afterwards, so later examples see the original.)

```
$ scripts/watchover forwarder --name /governance/events/pki-slack --type slack --preset certificates
↪  --items-source '/pki/*' --runner periodic --every 24 --write terraform/model/governance.yaml && grep -n
↪  -A3 'pki-slack' terraform/model/governance.yaml
added /governance/events/pki-slack to terraform/model/governance.yaml
59:  /governance/events/pki-slack:
60-    type: slack
61-    event_types:
62-    - certificate-pending-expiration

# provide these keys in TF_VAR_event_forwarder_secrets (CI secret, JSON map):
#   governance_events_pki_slack_url
```

exit code 0

Plain HCL plus an import block for a forwarder that already exists:

```
$ scripts/watchover forwarder --name /governance/events/siem --type webhook --url
↪  https://siem.example.com/akeyless --auth-type bearer-token --preset critical --auth-method-source '/*'
↪  --format hcl --import-existing
resource "akeyless_event_forwarder_webhook" "governance_events_siem" {
  name                               = "/governance/events/siem"
  event_types                        = ["certificate-expired", "certificate-provisioning-failure",
  ↪  "auth-method-expired", "rotated-secret-failure", "dynamic-secret-failure", "multi-auth-failure",
  ↪  "uid-rotation-failure", "gateway-inactive"]
  runner_type                        = "immediate"
  auth_methods_event_source_locations = ["/*"]
  url                                = "https://siem.example.com/akeyless"
  auth_type                          = "bearer-token"
  auth_token                         = var.event_forwarder_secrets["governance_events_siem_token"]
}

# imports.tf - the forwarder exists already; Terraform adopts it on the next apply
import {
  to = akeyless_event_forwarder_webhook.governance_events_siem
  id = "/governance/events/siem"
}

# needs: variable "event_forwarder_secrets" { type = map(string), sensitive = true } (already declared in
↪  terraform/main.tf)

# provide these keys in TF_VAR_event_forwarder_secrets (CI secret, JSON map):
#   governance_events_siem_token
```

exit code 0

ServiceNow with JWT authentication, created disabled:

```
$ scripts/watchover forwarder --name /governance/events/snow --type service_now --host
↪  example.service-now.com --auth-type jwt --client-id snow-client --user-email akeyless@example.com
↪  --preset rotation --items-source '/*' --disabled
# add under event_forwarders: in terraform/model/governance.yaml
```

```
  /governance/events/snow:
    type: service_now
    event_types:
    - next-automatic-rotation
    - rotated-secret-success
    - rotated-secret-failure
    - rotation-usage
    runner_type: immediate
    enable: false
    items_event_source_locations:
    - /*
    host: example.service-now.com
    auth_type: jwt
    client_id: snow-client
    user_email: akeyless@example.com
    client_secret_var: governance_events_snow_client_secret
    app_private_key_var: governance_events_snow_app_private_key

# provide these keys in TF_VAR_event_forwarder_secrets (CI secret, JSON map):
#   governance_events_snow_client_secret
#   governance_events_snow_app_private_key
```

exit code 0

Invalid input is rejected before Terraform runs (exit code 2):

```
$ scripts/watchover forwarder --name /governance/events/ops --type teams --preset operations --items-source
↪  '/*'
error: teams needs --gateway-source (provider 2.0.5 requires gateway_event_source_locations)
```

exit code 2

9.6 `watchover conjur`

Translate Conjur policy into model roles + identity bindings. *Offline.*

```
Translate CyberArk Conjur policy (RBAC) into the akeyless-watchover YAML model.

Akeyless "Automatic Migration" (Gateway, type conjur) copies *secret values* only.
Conjur's authorization model (policies, hosts, layers, groups, permits, grants) has
to be rebuilt as Akeyless roles - this script does the mechanical part:

  Conjur                              Akeyless model
  ----------------------------------  ------------------------------------------------
  !variable  <policy>/<id>            item  <prefix>/<policy>/<id>
  !permit  privileges on a variable   rule on that item path:
      execute (fetch value)  -> read      read   (+ list implied)
      read    (see metadata) -> list      list
      update  (set value)    -> update    update
  role with permits (layer/group/     one Akeyless role  <role_prefix>conjur/<kind>-<id>
      host/user/policy)
  !grant  role: A, member: B          no role nesting in Akeyless -> B is bound to A's
                                      Akeyless role too (flattened, transitive)
  host / user identities              bindings report: which Akeyless roles each Conjur
                                      identity needs; YOU map it to an auth method +
                                      sub-claims (e.g. authn-jwt -> OAuth2/JWT auth method)

Supported policy statements: !policy (id, body), !variable, !host, !layer, !group, !user,
!webservice, !permit, !grant, !deny (reported, not translated). Annotations are ignored.

Usage:
  python3 scripts/conjur2akeyless.py policy-root.yml [more.yml ...]       --item-prefix /conjur
  ↪  --role-prefix /governance/roles/ --out export/conjur
Writes <out>/roles/conjur.yaml, <out>/bindings.yaml and <out>/REPORT.md.
```

```
VERIFY: where the Gateway migration places each Conjur variable (target-location + Conjur
variable id is assumed here) - compare with the migrated item names before applying.
```

Options.

Option	Value	Required / default	Description
`policy`	`POLICY...`	**required**	Conjur policy YAML files (loaded at the root namespace in the given order)
`--item-prefix`	`ITEM_PREFIX`	default /conjur	Akeyless folder the Gateway migration imports into
`--role-prefix`	`ROLE_PREFIX`	default /governance/roles/	prefix of the generated Akeyless role names
`--out`	`OUT`	**required**	output directory (roles/conjur.yaml, bindings.yaml, REPORT.md)

Usage patterns.

```
scripts/watchover conjur root.yml prod.yml --item-prefix /conjur --out export/conjur
```

Example execution

Translate a Conjur policy:

```
$ scripts/watchover conjur scripts/tests/fixtures/conjur.yml --item-prefix /conjur --out export/conjur
2 roles, 2 identities, 1 note(s) -> export/conjur

                                                                                exit code 0
```

The translated roles and the identity bindings to complete:

```
$ ls export/conjur/roles && sed -n '1,25p' "$(ls -d export/conjur/roles/* | head -n 1)" && sed -n '1,15p'
↪  export/conjur/bindings.yaml
conjur.yaml
roles:
  /governance/roles/conjur/group-payments-admins:
    allow_external_auth_methods: true
    description: Translated from Conjur group 'payments-admins'
    rules:
    - capabilities:
      - list
      - update
      path: /conjur/prod/payments/db/password
  /governance/roles/conjur/layer-prod-payments:
    allow_external_auth_methods: true
    description: Translated from Conjur layer 'prod/payments'
    rules:
    - capabilities:
      - list
      - read
      path: /conjur/prod/payments/api-key
    - capabilities:
      - list
      - read
      path: /conjur/prod/payments/db/password
bindings:
  host:prod/payments/checkout:
    akeyless_roles:
```

```
    - /governance/roles/conjur/layer-prod-payments
    auth_method: TODO (e.g. /governance/auth/<idp-or-platform>)
    sub_claims: TODO (claims identifying this workload/user)
  user:alice:
    akeyless_roles:
    - /governance/roles/conjur/group-payments-admins
    auth_method: TODO (e.g. /governance/auth/<idp-or-platform>)
    sub_claims: TODO (claims identifying this workload/user)
```

exit code 0

9.7 `watchover account-settings`

Apply account_settings_cli (settings the provider cannot manage) with the CLI. *Calls Akeyless.*

```
Apply the account settings that Terraform provider 2.0.5 cannot manage.

Reads `account_settings_cli` from model/governance.yaml. Keys are the flag names of
`akeyless update-account-settings` (without the leading --):
  * lists  -> the flag is repeated once per value   (--bound-ips a --bound-ips b)
  * bools  -> --flag=true / --flag=false
  * others -> --flag=value
`update-account-settings` is admin-only and idempotent (same input -> same state).

Usage:
  python3 scripts/akl_account_settings.py --governance terraform/model/governance.yaml [--dry-run]
  ↪  [--profile P]
```

Options.

Option	Value	Required / default	Description
`--governance`	`GOVERNANCE`	**required**	path to model/governance.yaml (reads account_settings_cli)
`--profile`	`PROFILE`	optional	akeyless CLI profile of an account admin
`--token`	`TOKEN`	optional	Akeyless token (default: $AKEYLESS_TOKEN, e.g. from 'watchover token')
`--akeyless`	`AKEYLESS`	default akeyless	path to the akeyless CLI binary
`--dry-run`	—	optional	print the update-account-settings command, do not run it

Environment variables.

`AKEYLESS_TOKEN`	token used when --token is not given

Usage patterns.

```
scripts/watchover account-settings --governance terraform/model/governance.yaml --dry-run
AKEYLESS_TOKEN=$(scripts/watchover token) scripts/watchover account-settings --governance
↪  terraform/model/governance.yaml
```

Example execution

Dry run: print the CLI command for the settings Terraform cannot manage:

```
$ scripts/watchover account-settings --governance terraform/model/governance.yaml --dry-run
+ akeyless update-account-settings --allowed-client-type=cli --allowed-client-type=ui
↪  --allowed-client-type=sdk --bound-ips=10.0.0.0/8 --lock-allowed-client-type=true --lock-bound-ips=true
↪  --max-rotation-interval=90 --max-rotation-interval-enable=true --usage-event-enable=true
↪  --usage-event-interval=90 --usage-event-object-type=item

                                                                                exit code 0
```

Apply them with an admin CLI profile:

```
$ scripts/watchover account-settings --governance terraform/model/governance.yaml --profile admin
+ akeyless update-account-settings --allowed-client-type=cli --allowed-client-type=ui
↪  --allowed-client-type=sdk --bound-ips=10.0.0.0/8 --lock-allowed-client-type=true --lock-bound-ips=true
↪  --max-rotation-interval=90 --max-rotation-interval-enable=true --usage-event-enable=true
↪  --usage-event-interval=90 --usage-event-object-type=item --profile admin
Account settings were updated successfully

                                                                                exit code 0
```

9.8 `watchover token`

CI: exchange the pipeline JWT for an Akeyless token. *Calls Akeyless.*

```
Exchange a CI OIDC/JWT for a short-lived Akeyless token (REST POST /auth, access-type jwt). Prints
↪  the token.
```

Options.

Option	Value	Required / default	Description
`--api`	`API`	optional	Akeyless API (default: $AKEYLESS_API or https://api.akeyless.io)
`--access-id`	`ACCESS_ID`	optional	access ID of the CI auth method (default: $AKEYLESS_ACCESS_ID)
`--jwt`	`JWT`	optional	the CI-issued JWT (default: $CI_JWT; prefer the env var - arguments are visible in the process list)

Environment variables.

`AKEYLESS_API`	API base URL
`AKEYLESS_ACCESS_ID`	CI auth method access ID
`CI_JWT`	CI-issued JWT

Usage patterns.

```
CI_JWT=$JWT AKEYLESS_ACCESS_ID=p-xxxx scripts/watchover token
```

Example execution

Exchange the CI-issued JWT for an Akeyless token (printed on stdout): (Shown as you would run it; executed against the local stand-in endpoint.)

```
$ CI_JWT=$AKEYLESS_JWT AKEYLESS_ACCESS_ID=p-ci0demo scripts/watchover token
t-demo-0b3f9c
```

exit code 0

Missing settings stop the command before any request is made: (Shown as you would run it; executed against the local stand-in endpoint.)

```
$ scripts/watchover token
./scripts/akl_ci_token.sh: line 10: AKEYLESS_ACCESS_ID: set AKEYLESS_ACCESS_ID
```

exit code 1

9.9 `watchover build-skill`

Package the skill ZIP. *Offline.*

```
Build dist/akeyless-watchover.zip (SKILL.md + references/ + template/) for upload to Claude or
↪  unzip into ~/.claude/skills/.
```

No options.

Usage patterns.

```
scripts/watchover build-skill
```

Example execution

Package the skill for upload:

```
$ scripts/watchover build-skill && unzip -l dist/akeyless-watchover.zip | tail -n 1
built dist/akeyless-watchover.zip (156K)
   337065                    167 files
```

exit code 0

9.10 `watchover help`

Help in the terminal. `-h/--help` after any command shows that command; `watchover` with no argument is the same as `watchover help`.

Options.

Option	Value	Required / default	Description
`topic`	`TOPIC`	**required**	a kit command name or 'skill'; omitted = overview
`--markdown`	–	optional	print the whole usage guide as Markdown (docs/usage-guide.md)

Example execution

Overview:

```
$ scripts/watchover help
watchover 1.4.0 - akeyless-watchover kit

Usage: scripts/watchover <command> [options]

Commands:
  test              Run every offline check (what CI runs).
  simulate          Offline: evaluate access_expectations.yaml against the model.
  verify            Live: compare expectations with describe-permissions run as each persona's CLI profile.
  ↪  [needs Akeyless]
  export            Export existing config and translate it: YAML model + import blocks, or plain Terraform
  ↪  (--hcl).  [needs Akeyless]
  forwarder         Scaffold a new event forwarder (model entry or HCL); --import-existing adopts one by
  ↪  name.
  conjur            Translate Conjur policy into model roles + identity bindings.
  account-settings  Apply account_settings_cli (settings the provider cannot manage) with the CLI.  [needs
  ↪  Akeyless]
  token             CI: exchange the pipeline JWT for an Akeyless token.  [needs Akeyless]
  build-skill       Package the skill ZIP.
  help              This help. 'help <command>', 'help skill', 'help --markdown'.
  version           Print the version.

Skill commands (type them to Claude, e.g. '/akeyless-watchover help'):
  help, design, change, review, pipeline, import, conjur, pki, harden, gateway, verify, cli, explain

Full guide: docs/usage-guide.md
```

exit code 0

The skill commands:

```
$ scripts/watchover help skill
Skill commands - type them to Claude.
Claude Code: '/akeyless-watchover <command> [args]'.  Claude (web/desktop): 'akeyless-watchover <command>
↪  [args]'
or simply describe the task; the skill also triggers without a command.

  help [command]
      Show this guide, or the details of one skill command.
      delivers: The command table, parameters and examples.
      e.g. /akeyless-watchover help design

  design <diagram | description of BUs, apps, environments, personas>
      Translate an organisation/structure diagram into Akeyless configuration.
      delivers: Path tree, access matrix, YAML model (roles, auth methods, folders), access expectations,
      ↪  plan expectations, CLI equivalent.
      e.g. /akeyless-watchover design <paste or attach the diagram>

  change <access request in plain words>
      Change existing access with the smallest possible model diff.
      delivers: Diff of model/roles/<bu>.yaml (and auth_methods.yaml if needed), new expectations, who must
      ↪  approve (CODEOWNERS).
      e.g. /akeyless-watchover change give fraud-analysts read on /payments/+/prod/*

  review <model files, merge-request diff or plan output>
      Review a proposed change for least privilege, guard violations and blast radius.
```

```
      delivers: Findings by severity, failing guards, missing expectations, plan red flags (replacements).
      e.g. /akeyless-watchover review <paste the plan>

  pipeline [github | gitlab] [repository]
      Set up the governance repository: bootstrap identity, CI, CODEOWNERS, state.
      delivers: bootstrap.tfvars, CI file, variables to create, step-by-step order.
      e.g. /akeyless-watchover pipeline gitlab group/akeyless-governance

  import [what to export: everything / a type / one user's view] [export directory or REPORT.md]
      Export existing config (RBAC, auth methods incl. OIDC, folders, custom fields, key policies, PKI
      ↪  issuers, account settings) and translate it into Terraform.
      delivers: The exact `watchover export` command (model or --hcl mode, filters), then resolution of
      ↪  every REPORT.md item and the import checklist until the plan shows imports only.
      e.g. /akeyless-watchover import the OIDC auth method and everything alice@example.com can reach

  conjur <Conjur policy YAML>
      Migrate from CyberArk Conjur: secrets via the Gateway, RBAC via translation.
      delivers: Translated roles, bindings to fill (auth method + sub-claims), migration resource, cut-over
      ↪  plan.
      e.g. /akeyless-watchover conjur <attach policy files>

  pki <certificate requirements>
      Design private/public CA issuers and who may request certificates.
      delivers: pki.yaml (CA keys, issuers), requester roles, expiry forwarder, CLI to issue/revoke.
      e.g. /akeyless-watchover pki internal TLS for *.payments.internal, 90 days, auto-renew

  harden [current settings or concerns]
      Account-level hardening.
      delivers: governance.yaml: account_settings (Terraform), account_settings_cli (CLI), custom fields,
      ↪  key policies, event forwarders.
      e.g. /akeyless-watchover harden

  gateway <who administers which Gateway, SIEM target>
      Gateway admin plane and audit-log forwarding.
      delivers: gateway.yaml (allowed_access, log_forwarding), gateway_api setup.
      e.g. /akeyless-watchover gateway platform admins full, PKI team targets only, Splunk

  verify [persona | path]
      Prove who can access what.
      delivers: access_expectations.yaml checks, `watchover simulate` / `watchover verify` commands,
      ↪  Impersonate As steps.
      e.g. /akeyless-watchover verify can the checkout pods read refunds prod?

  cli <task>
      Exact Akeyless CLI commands for a task (break-glass, bootstrap, one-off checks).
      delivers: Commands checked against the CLI reference, with the equivalent model change.
      e.g. /akeyless-watchover cli create a temporary read rule for 60 minutes

  explain <topic>
      Explain an Akeyless concept precisely (sub-claims, deny, wildcards, anti-escalation...).
      delivers: A short answer with the verified facts and VERIFY caveats.
      e.g. /akeyless-watchover explain should this run through the API or the CLI?
```

exit code 0

One command (same as `scripts/watchover simulate --help`):

```
$ scripts/watchover help simulate
watchover simulate - Offline: evaluate access_expectations.yaml against the model.

Offline access check of the YAML model against expected access (no Akeyless calls).

Needs an Akeyless account: no

Parameters:
  --model                   MODEL          required              model directory (terraform/model)
  --expectations            EXPECTATIONS   required              access_expectations.yaml
  --explain                 -              optional              print the matching rules for every check,
  ↪  not only failures

Examples:
```

```
  scripts/watchover simulate --model terraform/model --expectations terraform/model/access_expectations.yaml
  ↪ --explain
```

exit code 0

The Markdown guide (docs/usage-guide.md):

```
$ scripts/watchover help --markdown | head -n 12
<!-- GENERATED by `scripts/watchover help --markdown` - edit scripts/watchover.py, not this file. -->
# akeyless-watchover usage guide (v1.4.0)

Two ways to use akeyless-watchover:

1. **Talk to Claude** with the skill installed: use a *skill command* (below) or just describe the task.
2. **Run the kit** in your governance repository with `scripts/watchover <command>` (Python 3.10+, `pyyaml`;
   Terraform >= 1.7 or OpenTofu >= 1.8; the `akeyless` CLI for live commands).

`scripts/watchover help` prints everything in this guide in the terminal.

## 1. Skill commands (Claude)
```

exit code 0

9.11 `watchover version`

Prints the kit version (also `--version`). No options.

Example execution

```
$ scripts/watchover version
1.4.0
```

exit code 0

Chapter 10

YAML model reference

All files live in `terraform/model/` (override with `var.model_dir`). Every file is optional except what you use. Object names are absolute paths (leading `/`, no trailing `/`). Values ending in `_var` are **names of keys** in a sensitive Terraform variable, never secrets.

10.1 `roles/<any>.yaml` - access roles (one file per BU, protected by CODEOWNERS)

```
roles:
  /governance/roles/<bu>/<role>:
    description: text
    delete_protection: true            # optional
    allow_root_scope: false            # required true for '/*' rules beyond list/deny
    allow_external_auth_methods: false # allow associations to auth methods not in
    ↪  auth_methods.yaml
    # management plane (see rbac-model.md for allowed values)
    audit_access: scoped
    analytics_access: own
    gw_analytics_access: none
    sra_reports_access: none
    usage_reports_access: none
    event_center_access: scoped
    event_forwarders_access: none
    event_forwarders_name: [/governance/events/x]
    reverse_rbac_access: scoped
    isi_access: none
    unlock_secrets: none
    approve_access_request: scoped
    rules:                             # AUTHORITATIVE - the full list of the role's rules
      - path: /payments/+/prod/*
        capabilities: [read, list]
        type: item-rule                # default; target-rule | role-rule | auth-method-rule |
        ↪  sra-rule | ara-rule*
    associations:
      - auth_method: /governance/auth/sso-entra-oidc
        sub_claims:                    # key: string "a,b" or list [a, b]  (OR within a key, AND
        ↪  across keys)
          groups: [grp-akl-payments-admins]
        case_sensitive: true           # default true
```

* `ara-rule` only with `-var allow_ara_rules=true`.

10.2 `auth_methods.yaml`

```
auth_methods:
  /governance/auth/<name>:
    type: oidc        # api_key | oidc | saml | oauth2 | aws_iam | azure_ad | k8s |
    ↪  universal_identity
    # common to all types
```

```
    description: text
    access_expires: 0             # unix timestamp, 0 = never
    bound_ips: [10.0.0.0/8]
    gw_bound_ips: [10.1.0.0/16]
    force_sub_claims: true        # recommended for shared IdP/CI methods
    jwt_ttl: 60                   # minutes
    audit_logs_claims: [email]
    allowed_client_type: [cli, ui] # cli, ui, gateway-admin, sdk, mobile, extension
    product_type: [sm]            # sm, sra, pm, dp, ca
    expiration_event_in: ["30"]
    delete_protection: true
```

Type-specific keys:

type	keys
oidc	`unique_identifier`*, `issuer`, `client_id`, `client_secret_var`, `audience`, `allowed_redirect_uri`, `required_scopes`, `required_scopes_prefix`, `subclaims_delimiters`
saml	`unique_identifier`*, `idp_metadata_url`, `idp_metadata_xml_data`, `allowed_redirect_uri`, `subclaims_delimiters`
oauth2	`unique_identifier`*, `jwks_uri`, `jwks_json_data`, `issuer`, `audience`, `bound_client_ids`, `subclaims_delimiters`
aws_iam	`bound_aws_account_id`*, `bound_arn`, `bound_role_name`, `bound_role_id`, `bound_user_name`, `bound_user_id`, `bound_resource_id`, `sts_url`
azure_ad	`bound_tenant_id`*, `bound_spid`, `bound_group_id`, `bound_sub_id`, `bound_rg_id`, `bound_resource_id`, `bound_resource_names`, `bound_resource_types`, `bound_providers`, `audience`, `issuer`, `jwks_uri`
k8s	`bound_namespaces`, `bound_sa_names`, `bound_pod_names`, `audience`, `gen_key`, `public_key` (also needs a Gateway K8s auth config)
universal_identity	`ttl`, `deny_inheritance`, `deny_rotate`
api_key	- (the access key ends up in state; prefer identity-based methods)

* = required. The CI identity Terraform runs as is **not** declared here (see `bootstrap/`).

10.3 `folders.yaml`

```
folders:
  /payments:
    description: text
    delete_protection: true
    tags: [bu:payments]
```

10.4 `governance.yaml`

```
custom_fields:                    # mandatory metadata on secrets
  <id>:
    name: owner                   # default: the key
    object_type: STATIC_SECRET    # STATIC_SECRET | DYNAMIC_SECRET | ROTATED_SECRET
```

```
    object: <optional>
    required: true

key_policies:
  /payments/*:                    # path
    allowed_algorithms: [AES256GCM, RSA4096]
    allowed_key_types: [dfc]      # dfc, classic-key
    allowed_key_names: []
    max_rotation_interval_days: 365
    object_types: [items]         # items, targets

account_settings:                 # Terraform-managed subset (provider 2.0.5)
  # company_name, default_share_link_ttl_minutes, default_versioning, dynamic_secret_max_ttl,
  # dynamic_secret_max_ttl_enable, enable_ai_quorum, enable_item_sharing,
  ↪  enable_password_expiration,
  # hide_personal_folder, hide_static_password, invalid_characters, item_locking_enabled,
  # items_deletion_protection, jwt_ttl_default, jwt_ttl_max, jwt_ttl_min, max_versions,
  # password_expiration_days, password_expiration_notification_days, password_length,
  # use_capital_letters, use_lower_letters, use_numbers, use_special_characters

account_settings_cli:             # applied by scripts/akl_account_settings.py
  # keys = update-account-settings flags without '--'; lists repeat the flag; bools ->
  ↪  =true/false
  bound-ips: [10.0.0.0/8]
  lock-bound-ips: true

event_forwarders:
  /governance/events/<name>:
    type: email                   # email | webhook | slack | teams | service_now
    event_types: [certificate-pending-expiration, ...]   # validated list, see main.tf
    runner_type: immediate        # immediate | periodic (teams: required)
    every: 24                     # hours, periodic only
    enable: true
    items_event_source_locations: ["/*"]
    targets_event_source_locations: []
    auth_methods_event_source_locations: []
    gateways_event_source_locations: []     # teams uses gateway_event_source_locations
    ↪  (required)
    key: /optional/protection-key
    # email:   email_to (list|string), include_error, override_url
    # webhook: url, auth_type (user-pass|bearer-token|certificate), username, password_var,
    ↪  auth_token_var,
    #          client_cert_data_var, private_key_data_var, server_certificates_var
    # slack:   url_var        teams: url_var
    # service_now: host, auth_type (user-pass|jwt); user-pass: admin_name, admin_pwd_var;
    #          jwt: client_id, user_email, client_secret_var, app_private_key_var
```

*`_var` values are keys of the `event_forwarder_secrets` variable (`TF_VAR_event_forwarder_secrets`, a JSON map kept in CI secrets); secrets never go in the model. Guards: valid type and event types, `every` with `runner_type: periodic`, `url_var` for slack/teams, `gateway_event_source_locations` + `runner_type` for teams.

Akeyless has no operation that lists event forwarders, so `watchover export` cannot read them. Write new ones with `watchover forwarder` (validated entry, `--write` inserts it here, `--format hcl` for plain Terraform); adopt an existing one whose name you know with `--import-existing` (import ID = name). Presets: `certificates`, `auth-methods`, `rotation`, `secrets`, `access-requests`, `operations`, `critical` (see `watchover help forwarder`).

10.5 `pki.yaml`

```
ca_keys:                            # private CA signer keys (DFC RSA, self-signed CA
↪  certificate)
  /pki/ca/<name>:
    alg: RSA4096                    # default RSA4096
    certificate_ttl_days: 3650      # required
    path_len: 0                     # optional, adds pathlen to basicConstraints
    subject: { countryName: ES, organizationName: X, commonName: X Root CA }
    delete_protection: true         # default true
issuers:
  /pki/issuers/<name>:
    ttl_seconds: 7776000            # required
    signer_key_name: /pki/ca/<name> # private CA  (exactly one of signer_key_name / ca_target)
    ca_target: /targets/...         # public CA (requires gw_cluster_url)
    gw_cluster_url: https://gw:8000 # required for ca_target, private CRL/OCSP, ACME
    allowed_domains: [a.example.com]
    allowed_uri_sans: []
    allowed_ip_sans: []
    allow_subdomains: true
    allow_any_name: false
    disable_wildcards: false
    not_enforce_hostnames: false
    not_require_cn: false
    server_flag: true
    client_flag: false
    code_signing_flag: false
    key_usage: [DigitalSignature, KeyEncipherment]
    critical_key_usage: true
    organizations: X
    organizational_units: Y
    country: ES
    locality: Madrid
    province: Madrid
    max_path_len: -1
    destination_path: /pki/certs/<x>   # required for auto_renew and CRL
    protect_certificates: true
    auto_renew: true
    scheduled_renew_days: 30
    expiration_event_in_days: [30, 7]
    create_public_crl: false
    create_private_crl: false
    create_public_ocsp: false
    create_private_ocsp: false
    enable_acme: false
    allowed_extra_extensions: { "1.2.3": ["value"] }
    delete_protection: true            # default true
    tags: []
```

10.6 `gateway.yaml` (needs `var.gateway_api`)

```
allowed_access:
  <name>:
    auth_method: /governance/auth/<name>   # or access_id: p-xxxx (exactly one)
    permissions: [admin]                    # validated list, see rbac-model.md
    sub_claims: { groups: [grp-x] }
    case_sensitive: true
    description: text
log_forwarding:                             # singleton per Gateway
  type: syslog                              # syslog | splunk
```

```
  # syslog: host*, network (tcp|udp), enable_tls, formatter (text|cef), output_format
  ↪  (text|json), target_tag, pull_interval
  # splunk: splunk_url*, splunk_token_var*, index, source, source_type, enable_tls, enable_batch,
  ↪  output_format, pull_interval
migrations:
  conjur:
    <name>:
      target_location: /conjur
      conjur_url: https://conjur.example.com
      conjur_account: myorg
      conjur_username: host/akeyless-migration
      conjur_api_key_var: conjur_api_key
      protection_key: /optional/key
```

10.7 access_expectations.yaml

```
personas:
  <persona>:
    auth_method: /governance/auth/<name>
    claims: { groups: [grp-x], email: [a@b.c] }   # claims the persona's token would carry
    profile: <cli-profile>                          # optional, for live verification
checks:
  - { persona: <persona>, path: /x/y, type: item, expect: [read, list] }  # type:
  ↪  item|target|role|auth-method
```

Chapter 11

Terraform reference implementation (`terraform/`)

Data-driven: governance lives in YAML, the HCL is generic. Tested offline with OpenTofu 1.10 and provider 2.0.5 (`validate`, `test` with a mocked provider); the repository CI also runs HashiCorp Terraform. Requires Terraform >= 1.7 or OpenTofu >= 1.8.

File	Manages
`versions.tf`	provider pin `~> 2.0`, JWT login, `akeyless.gateway` alias, endpoint variables
`main.tf`	model loading, **guards**, folders
`auth_methods.tf`	one resource block per auth method type, `prevent_destroy`
`roles.tf`	roles (authoritative rules) and role <-> auth method associations
`governance.tf`	custom fields, key policies, account settings subset, event forwarders
`pki.tf`	CA signer keys (DFC, self-signed CA cert) and PKI issuers
`gateway.tf`	Gateway allowed access, log forwarding (syslog/splunk), Conjur migration
`outputs.tf`	access IDs, role names

11.1 Inputs

Variable	Purpose
`pipeline_access_id`, `pipeline_jwt`	CI identity (bootstrap output) and CI-issued JWT
`akeyless_api`	default `https://api.akeyless.io`
`gateway_api`	Gateway API for gateway objects; `null` or `""` disables them
`auth_method_secrets`, `event_forwarder_secrets`, `gateway_secrets`	sensitive maps referenced by `*_var` keys
`naming`	`role_prefix` / `auth_method_prefix` (set to `""` to disable; `null` means the default)
`allow_ara_rules`	opt-in for `ara-rule`
`model_dir`	model location (tests point it to fixtures)

11.2 Guards (fail the plan, never the apply)

`terraform_data.model_guards` preconditions; every resource depends on it.

Guard	Fixture proving it
duplicate role across files	`dup`

Guard	Fixture proving it
absolute names, no trailing `/`	-
naming prefixes	`bad_prefix`
supported auth method types	-
supported rule types (ara opt-in)	`bad_rule_type`, `ara_allowed`
capabilities valid per rule type	`bad_capability`
deny rules contain only deny	`deny_mixed`
SRA: approval_authority without request_access	`sra_conflict`
`/*` beyond list/deny (or read on role/auth-method rules) needs `allow_root_scope`	`root_scope`, `root_allowed`
management-plane values	`bad_mgmt`
associations only to modelled auth methods (or `allow_external_auth_methods`)	`unknown_am`
`force_sub_claims` auth methods require sub-claims on every association	`missing_subclaims`
issuer: signer xor ca_target; auto_renew needs destination_path; CRL/OCSP/ACME/public CA need gw_cluster_url	`bad_issuer`
custom field object types, event forwarder types/events, teams requirements, periodic runner needs `every`, slack/teams need `url_var`	`bad_event`, `bad_periodic`, `slack_no_url`
gateway objects need `gateway_api`; valid gateway permissions; auth_method xor access_id; log forwarding type	`bad_gateway`, `gateway_objects_need_gateway_api`, `empty_gateway_api_is_unset`

`tests/guard_messages.sh` proves each negative fixture fails with the **intended** message (`expect_failures` alone only proves that *some* guard failed).

11.3 Provider facts this code relies on (verified in v2.0.5 source/schema)

1. `akeyless_role.rules` is authoritative; there is no per-rule resource.
2. `assoc_auth_method` inside `akeyless_role` is deprecated - associations are separate resources. The role only reads associations back when its own block is used, so there is no drift.
3. `name` is ForceNew on roles, auth methods, folders, issuers, keys, targets, static secrets; `role_name/am_name` on associations. Rename = destroy + create (new access ID for auth methods).
4. `delete_protection`: string on auth methods/roles/folders/static secrets/certificates, bool on `akeyless_pki_cert_issuer` and `akeyless_dfc_key`.
5. Not marked sensitive in the schema: `akeyless_auth_method_oidc.client_secret`, computed `akeyless_auth_method_k8s.private_key` - always feed through sensitive variables.
6. `jwt_login.jwt` must be set in HCL (validation fails otherwise, despite the `AKEYLESS_AUTH_JWT` env fallback in the provider code).
7. Env fallbacks: `AKEYLESS_GATEWAY`; without a login block `AKEYLESS_ACCESS_ID` + `AKEYLESS_ACCESS_KEY`.
8. Role names do not suppress leading-slash differences (auth methods and associations do).
9. `akeyless_account_settings` is partial and its destroy is a no-op.
10. `akeyless_pki_cert_issuer.ttl` is a string in seconds; `key_usage` default `DigitalSignature,KeyAgreement,KeyEncipherment`.
11. Import IDs: object name; associations `"<role>:<auth method>"` or `"<role>:<assoc_id>"`.

12. Refreshing `akeyless_static_secret` reads the secret value (keep values out of this state).

Chapter 12

Terraform variables and outputs

Generated from the `variable` and `output` blocks. Secrets are passed as `TF_VAR_<name>` environment variables from CI secrets (JSON for the maps), never in files.

12.1 Governance root module (`terraform/`)

Input variables

`model_dir`. default `"model"`.
Directory (relative to the root module) holding the YAML model.
Type: `string`

`auth_method_secrets`. default `{}` – *sensitive*.
Secrets referenced by *_var keys in model/auth_methods.yaml (e.g. OIDC client secrets). Inject from CI, never commit.
Type: `map(string)`

`event_forwarder_secrets`. default `{}` – *sensitive*.
Secrets referenced by *_var keys in model/governance.yaml event_forwarders (webhook passwords/tokens/certificates, Slack/Teams URLs, ServiceNow credentials).
Type: `map(string)`

`gateway_secrets`. default `{}` – *sensitive*.
Secrets referenced by *_var keys in model/gateway.yaml (e.g. Splunk token).
Type: `map(string)`

`naming`. default `{}`.
Naming policy enforced by the guards. Set a prefix to "" to disable that check (null falls back to the default).

```
object({
    role_prefix        = optional(string, "/governance/roles/")
    auth_method_prefix = optional(string, "/governance/auth/")
  })
```

`allow_ara_rules`. default `false`.
Allow rule type ara-rule. Not documented by provider 2.0.5 - enable only after verifying on your account.
Type: `bool`

`akeyless_api`. default `"https://api.akeyless.io"`.
Akeyless API endpoint for control-plane objects: https://api.akeyless.io (default) or a Gateway V2 API.
Type: `string`

`gateway_api`. default `null`.
Gateway V2 API (https://<gw>:8000/api/v2 or https://<gw>:8081) used for gateway_* resources. null = gateway objects disabled. One Gateway cluster per state/workspace.
Type: `string`

`pipeline_jwt`. **required** – *sensitive*.
CI OIDC/JWT token (export TF_VAR_pipeline_jwt in the pipeline).
Type: `string`

`pipeline_access_id`. **required**.
Access ID of the CI/CD JWT (OAuth2) auth method created by ../bootstrap.
Type: `string`

Outputs

`auth_method_access_ids`	Access IDs to hand to consumers (non-secret).
`role_names`	Managed access roles.
`association_count`	--

12.2 Bootstrap module (`bootstrap/`)

Input variables

`akeyless_api`. default `"https://api.akeyless.io"`.
_
Type: `string`

`ci`. **required**.
CI system trusted to run Terraform.

```
object({
    platform     = string                                  # "github" | "gitlab"
    repository   = string                                  # github: org/repo ; gitlab: group/project
    gitlab_url   = optional(string, "https://gitlab.com") # gitlab only
    audience     = optional(string, "akeyless")
    apply_ref    = optional(string, "refs/heads/main") # github: ref claim ; gitlab: branch name, e.g. "main"
    auth_name    = optional(string, "/governance/auth/ci-terraform")
    roles_prefix = optional(string, "/governance/roles/platform/")
  })
```

`managed_scopes`. default `{}`.
Paths the apply identity may administer. Keep '/*' unless the account hosts objects that must stay out of reach of this pipeline.

```
object({
    items        = optional(string, "/*")
    targets      = optional(string, "/*")
    roles        = optional(string, "/*")
    auth_methods = optional(string, "/*")
  })
```

Outputs

`pipeline_access_id`	Set as CI variable AKEYLESS_PIPELINE_ACCESS_ID (not secret).

Chapter 13

CLI playbook

Command style varies by family: `auth-method create <type>`, `target create <type>`, `rotated-secret create <type>`, `dynamic-secret create <type>`, `folder create`, while roles and PKI keep hyphenated verbs (`create-role`, `set-role-rule`, `create-pki-cert-issuer`). Older hyphenated auth forms (`create-auth-method-oidc`) still appear in docs. The installed CLI wins: `akeyless <command> -h`. Global flags: `--profile`, `--token`, `--uid-token`, `--json`, `--jq-expression`. Profiles: `~/.akeyless/profiles` (`akeyless configure`). Boolean-like flags are written `--flag=true`, accepted for bool and `[true/false]` string flags alike.

Install: `curl -o akeyless https://akeyless-cli.s3.us-east-2.amazonaws.com/cli/latest/cli-linux-amd64 && chmod +x akeyless` (or `brew tap akeylesslabs/tap`).

13.1 RBAC

```
akeyless create-role --name /governance/roles/payments/team-admin --description "..."
↪  --audit-access scoped
akeyless set-role-rule --role-name /governance/roles/payments/team-admin \
  --path "/payments/+/nonprod/*" --capability read --capability list --capability create
  ↪  --capability update --capability delete
akeyless set-role-rule --role-name /governance/roles/payments/team-admin --path
↪  "/payments/+/prod/break-glass/*" --capability deny
akeyless set-role-rule --role-name <role> --path "/governance/roles/*" --rule-type role-rule
↪  --capability read --capability list
akeyless set-role-rule --role-name <role> --path "<path>" --capability read --ttl 60   #
↪  temporary (minutes)
akeyless delete-role-rule --role-name <role> --path "<path>" --rule-type item-rule
akeyless assoc-role-am --role-name <role> --am-name /governance/auth/sso-entra-oidc \
  --sub-claims groups=grp-akl-payments-admins --case-sensitive=true
akeyless update-assoc --assoc-id <assoc-id> --sub-claims groups=grp-a,grp-b
akeyless delete-assoc --assoc-id <assoc-id>
akeyless get-role --name <role> --json          # rules + associations (assoc IDs)
akeyless list-roles --json
akeyless update-role --name <role> --new-name <new>   # rename (Terraform cannot)
```

Temporary rules on a Terraform-managed role are removed by the next apply (rules are authoritative). For just-in-time access prefer `request-access` + roles with `approve_access_request`.

13.2 Auth methods

```
akeyless auth-method create oidc --name /governance/auth/sso-entra-oidc --issuer <url> \
  --client-id <id> --client-secret <secret> --unique-identifier email --force-sub-claims=true
  ↪  --jwt-ttl 60
akeyless auth-method create oauth2 --name /governance/auth/ci-terraform \
  --jwks-uri https://token.actions.githubusercontent.com/.well-known/jwks --unique-identifier
  ↪  repository
akeyless auth-method create aws-iam --name <n> --bound-aws-account-id <acct> --bound-role-name
↪  <role>
akeyless auth-method create azure-ad --name <n> --bound-tenant-id <tenant>
akeyless list-auth-methods --json ; akeyless get-auth-method --name <n> --json
```

Common flags: `--access-expires`, `--bound-ips`, `--gw-bound-ips`, `--force-sub-claims`, `--jwt-ttl`, `--audit-logs-claims`, `--allowed-client-type`, `--product-type`, `--expiration-event-in`, `--delete-protection`.

13.3 Inspection

```
akeyless describe-permissions --path /payments/checkout/prod/db/app --type item   # MY
↪  permissions on the path
akeyless reverse-rbac --path <path> --type item|am|role|target|sra|ara             # who reaches
↪  the path
akeyless describe-sub-claims [--profile <p>]
akeyless list-items --path /payments --json
akeyless move-objects --source /old/folder --target /new/folder                    # folders only
```

13.4 Account settings (CLI-only part)

`scripts/akl_account_settings.py` turns `account_settings_cli` in `governance.yaml` into one idempotent `update-account-settings` call (admin-only). It refuses keys the Terraform resource manages and unknown flags.

```
python3 scripts/akl_account_settings.py --governance terraform/model/governance.yaml --dry-run
AKEYLESS_TOKEN=$(AKEYLESS_ACCESS_ID=p-xxx CI_JWT=$JWT scripts/akl_ci_token.sh) \
  python3 scripts/akl_account_settings.py --governance terraform/model/governance.yaml
```

Test `lock-bound-ips` from a non-allowed network before locking it in production.

13.5 Gateway

```
akeyless gateway-create-allowed-access --name pki-admins --access-id <p-xxxx> \
  --sub-claims groups=grp-akl-pki --permissions general,targets --gateway-url 'https://<gw>:8000'
akeyless gateway-create-migration --type conjur --name conjur-prod --target-location /conjur \
  --conjur-url https://conjur --conjur-account myorg --conjur-username <u> --conjur-api-key <k> \
  --gateway-url 'https://<gw>:8000'
```

13.6 PKI

```
akeyless create-dfc-key --name /pki/ca/root --alg RSA4096 --generate-self-signed-certificate true
↪  \
  --certificate-ttl 3650 --conf-file-path ./csr.conf --certificate-format pem
akeyless create-pki-cert-issuer --name /pki/issuers/x --signer-key-name /pki/ca/root --ttl 30d \
  --destination-path /pki/certs/x --expiration-event-in 30
akeyless get-pki-certificate --cert-issuer-name /pki/issuers/x --csr-file-path ./my.csr --outfile
↪  cert.pem
akeyless revoke-certificate --name <cert item> --version <n>
```

Chapter 14

Akeyless REST API or CLI?

14.1 Short answer

They are two clients of **the same operations**. Neither one can read or export configuration that the other can't. Choose by where the command runs and how it authenticates, not by what it can reach.

14.2 Evidence

- The Akeyless OpenAPI specification (Go SDK v5.0.x) has 666 operations, 665 of them `POST`. 659 of their request schemas are declared in Akeyless's Go package `go/src/client/commands`, which holds the CLI command definitions. The REST API and the CLI flags come from the same structs, so a CLI flag is the API field in kebab-case (`--role-name` = `role-name`).
- `akeyless <command> --json` prints the response document of the matching API operation. This repository proves the equivalence: the exporter reads the same fixture account through the CLI, through REST and from saved JSON, and a test requires byte-identical output (`test_transports_produce_identical_output`).
- The Terraform provider uses the REST API (Go SDK). Anything Terraform reads, the API returns.

14.3 Differences that do matter

	CLI	REST API
Authentication	profiles in `~/.akeyless/profiles`, interactive SAML/OIDC browser login (`akeyless configure`)	send `token` in every JSON body; get one from `POST /auth` (any access type, e.g. `jwt` in CI)
Installation	binary (S3 download or brew) that runners must be allowed to fetch	HTTPS to `api.akeyless.io` or your Gateway only
Command names	`folder get`, `list-groups`, `auth-method create oidc`, `policy list`	`/folder-get`, `/list-group`, `/auth-method-create-oidc`, `/policy-list`
Gateway operations	`--gateway-url https://<gw>:8000` on each command	base URL `https://<gw>:8000/api/v2`
Client-side helpers	`set-role-rule --file-rules` (bulk JSON; the API takes one rule per call), `--jq-expression`, `connect` (SRA sessions), `agent`, `mcp`, `update`	none
Output	JSON with `--json`, text otherwise	always JSON

`/configure` and `/connect` also appear in the specification, because it's generated from the CLI commands. They're client workflows (profiles, SSH/RDP sessions): don't build on them over REST (VERIFY).

14.4 Decision rules

Situation	Use
Any persistent change (roles, auth methods, folders, issuers, settings...)	**Terraform** (REST via the provider), through the pipeline
A person exploring or exporting with their SSO login	**CLI** (`watchover export --transport cli --profile <sso-profile>`)
CI jobs: exports, drift reports, verification without the CLI binary	**REST** (`watchover token` -> `watchover export --transport api`)
Locked-down runners that can't download binaries	**REST**
Secure Remote Access sessions, the local agent, MCP	**CLI** (client-side features)
One-off bulk rule changes during an incident	CLI `set-role-rule --file-rules`, then codify in Git
An operation documented only in the API reference	**REST** (the CLI usually has it under a similar name: check `-h`)

14.5 Did the CLI unlock more exports?

Not by itself: the reads are the same. Going through the full list of read operations (76 `list-*`, `get-*` and `describe-*` calls) did show families the exporter wasn't using yet. They're now exported with `--hcl`:

Family	Read operation	Terraform resources
Targets	`list-targets`	`akeyless_target_*` (aws, azure, gcp, k8s and hashi_vault confirmed from the provider's constants; other types matched by name and marked VERIFY)
Groups (OIDC apps)	`list-groups` / `/list-group`	`akeyless_group`
Gateway	`gateway-get-config`	`akeyless_gateway_defaults`, `_cache`, `_remote_access`, `_log_forwarding_<type>`, `_migration_<type>`, `_allowed_access`

Still not exportable through either client:

- Event forwarders: `get-event-forwarder` reads one by name, and there is no list.
- Rotated and dynamic secret definitions: `rotated-secret-list` and `dynamic-secret-list` exist, but their producer types aren't yet mapped to provider resources.
- Secret values: excluded on purpose.

For ad-hoc SQL inventory queries outside Terraform, the community also publishes a Steampipe plugin (`akeyless-community/akeyless`, tables such as `akeyless_target` and `akeyless_gateway`).

14.6 In this kit

```
## person with an SSO CLI profile
scripts/watchover export --out export/ --transport cli --profile my-sso
## CI, no CLI binary
export AKEYLESS_TOKEN=$(CI_JWT=$JWT AKEYLESS_ACCESS_ID=p-xxxx scripts/watchover token)
scripts/watchover export --out export/ --transport api --hcl --types targets,groups,gateway \
  --gateway-api https://gw.example.com:8000/api/v2
```

Part IV

Operations

Chapter 15

Pipeline: bootstrap, CI/CD, drift and state

15.1 1. Bootstrap (once, by an Akeyless admin)

```
cd bootstrap
cp bootstrap.tfvars.example bootstrap.tfvars          # platform, repository, gitlab_url...
export AKEYLESS_ACCESS_ID=... AKEYLESS_ACCESS_KEY=... # admin API key (provider env fallback)
terraform init && terraform apply -var-file=bootstrap.tfvars
terraform output pipeline_access_id                    # -> CI variable
↪  AKEYLESS_PIPELINE_ACCESS_ID
```

Creates:

- OAuth2/JWT auth method `/governance/auth/ci-terraform` trusting GitHub Actions (`https://token.actions.githubusercontent.com`) or your GitLab (`<gitlab>/oauth/discovery/keys`), `force_sub_claims = true`, audience `akeyless`.
- `.../terraform-plan` role bound with `repository` (GitHub) / `project_path` (GitLab).
- `.../terraform-apply` role bound with repository **and** `ref=refs/heads/main` (GitHub) or `ref=main` + `ref_protected=true` (GitLab).

Keep bootstrap state separate and locked down. Delete the admin API key afterwards or keep it as break-glass in your PAM vault. **VERIFY:** the plan role's minimum read rights are not documented; extend it if a pull-request plan fails with an authorization error.

15.2 2. CI templates (`ci/`)

File	Use
`ci/github/akeyless-governance.yml`	copy to `.github/workflows/` in your governance repo
`ci/gitlab/.gitlab-ci.yml`	copy to the repo root (GitLab)
`ci/CODEOWNERS.example`	per-BU ownership of `model/roles/<bu>.yaml`

Jobs: **checks** (fmt, validate, tests, guard messages, simulator, script tests) -> **plan** on pull/merge requests -> **apply** on the default branch behind a protected environment, followed by `akl_account_settings.py` -> **drift** nightly (`plan -detailed-exitcode`, exit 2 = drift -> job fails).

Tokens: GitHub requests an OIDC token (`id-token: write`, audience `akeyless`); GitLab uses `id_tokens` with `aud: akeyless`. Terraform receives it as `TF_VAR_pipeline_jwt`; the CLI step exchanges it for an Akeyless token with `scripts/akl_ci_token.sh` (REST `POST /auth`, `access-type=jwt`) and passes `--token`.

Variables: `AKEYLESS_PIPELINE_ACCESS_ID`, optional `AKEYLESS_API`, `AKEYLESS_GATEWAY_API`, and the JSON maps `TF_VAR_auth_method_secrets`, `TF_VAR_event_forwarder_secrets`, `TF_VAR_gateway_secrets` (secrets/masked). Add your backend configuration.

15.3 3. State

Remote, encrypted, versioned, access-restricted. It contains API-key access keys, OIDC client secrets, K8s auth private keys, event-forwarder/webhook secrets, Splunk tokens and the Conjur API key (provider 2.0.5 has no write-only arguments). Treat the state bucket like a vault.

15.4 4. Drift

Because role rules are authoritative, console edits show up in the nightly plan and are reverted by the next apply. Tell administrators: ”change Akeyless through a merge request; console changes are temporary”. Emergency changes: make them, then raise the MR that codifies them.

Chapter 16

Testing and verification

Layer	Tool	Needs Akeyless?	Runs
Syntax/types	`terraform fmt -check`, `validate`	no	every PR
Model rules	`terraform test` (mocked provider, 19 runs)	no	every PR
Guard intent	`terraform/tests/guard_messages.sh`	no	every PR
Effective access (model semantics)	`scripts/akl_simulate.py`	no	every PR
Tooling + usage-guide drift	`pytest scripts/tests` (50 tests)	no	every PR
Bootstrap	`bootstrap/tests` (3 runs)	no	repo CI
Plan review	`terraform plan`	yes (plan identity)	every PR
Effective access (platform truth)	`scripts/akl_verify.py` / Console Impersonate As	yes	after apply
Drift	`terraform plan -detailed-exitcode`	yes	nightly

16.1 Offline access simulator

`akl_simulate.py` evaluates `access_expectations.yaml` against the model with the documented semantics (wildcards * and +, implied `list`, deny precedence, sub-claim OR/AND/globs/case, templated paths). Assumptions it makes - verify live once: `/a/*` does not match `/a` itself; explicit `AND`/`OR` sub-claim operators are evaluated left to right.

```
python3 scripts/akl_simulate.py --model terraform/model --expectations
↪  terraform/model/access_expectations.yaml --explain
```

16.2 Live verification

Give personas a CLI `profile` (test SSO user in the right group, test workload identity) and run:

```
python3 scripts/akl_verify.py --expectations terraform/model/access_expectations.yaml
```

It calls `describe-permissions --json` as each persona and compares `client_permissions` with the expected set. How the CLI reports "no access at all" is not documented: by default an error stops the run; after checking the behaviour, `--error-means-no-access` treats it as an empty set.

16.3 Running everything locally

One command: `scripts/watchover test` (or `--tool tofu`). Step by step:

```
(cd terraform && terraform init -backend=false && terraform validate && terraform test &&
↪  tests/guard_messages.sh terraform)
(cd bootstrap && terraform init -backend=false && terraform test)
pip install pyyaml pytest && python3 -m pytest -q scripts/tests
python3 scripts/akl_simulate.py --model terraform/model --expectations
↪  terraform/model/access_expectations.yaml
```

Chapter 17

Export existing configuration and translate it into Terraform

`watchover export` reads what already exists in an Akeyless account and translates it into Terraform. It only **reads**: nothing in Akeyless changes until you apply.

17.1 What is exported

`--types` value	Read with	Model mode (YAML)	`--hcl` mode (plain Terraform)
`roles`	`list-roles`, `get-role`	rules, management-plane access (decoded from `search-rule/reports-rule`... `/*`=all, `/self`=own, `/scoped`=scoped), associations + sub-claims	`akeyless_role`, `akeyless_associate_role_auth_method`
`auth-methods`	`list-auth-methods`, `get-auth-method`	full settings for OIDC, SAML, OAuth2/JWT, AWS IAM, Azure AD, K8s, Universal Identity, API key, plus bound IPs, JWT TTL, claims...	the above + GCP, LDAP, certificate, OCI, AliCloud, Kerberos, email
`folders`	`list-items` walk + `folder get`	explicit folders (description, tags, delete protection)	`akeyless_folder`
`custom-fields`	`account-custom-field list`	`governance.yaml` `custom_fields`	`akeyless_account_custom_field`
`key-policies`	`policy list`	`governance.yaml` `key_policies`	`akeyless_policy_keys`
`pki-issuers`	`list-items` walk + `describe-item`	reported (use `--hcl`)	`akeyless_pki_cert_issuer`
`account-settings`	`get-account-settings`	saved to raw/ (use `--hcl`)	`akeyless_account_settings`
`targets`	`list-targets`	reported (use `--hcl`)	`akeyless_target_*` (type mapping: confirmed for aws, azure, gcp, k8s, hashi_vault; others by name, VERIFY)
`groups`	`list-groups` (`/list-group`)	reported (use `--hcl`)	`akeyless_group`

`--types` value	Read with	Model mode (YAML)	`--hcl` mode (plain Terraform)
`gateway` (not in the default set)	`gateway-get-config`	reported (use `--hcl`)	`akeyless_gateway_defaults`, `_cache`, `_remote_access`, `_log_forwarding_<type>`, `_migration_<type>`, `_allowed_access` (with `provider = akeyless.gateway`)

Not exported: secret values, keys, rotated/dynamic secret definitions, event forwarders (no list operation).

17.2 Transport: CLI or REST

`--transport cli` (default) runs the `akeyless` CLI with `--profile` (SSO logins work). `--transport api` calls the REST API directly with `--token` / `$AKEYLESS_TOKEN`, with no binary needed. Both read the same operations and produce identical output (tested). See api-vs-cli.md. Gateway reads need `--gateway-url https://gw:8000` (CLI) or `--gateway-api https://gw:8000/api/v2` (REST).

The auth-method field mapping (e.g. Azure `bound_service_principal_ids` -> `bound_spid`, AWS `account_id` -> `bound_aws_account_id`) was taken from the provider 2.0.5 read code of every `akeyless_auth_method_*` resource.

17.3 Two modes

Model mode (default): for this repository's data-driven Terraform.

```
scripts/watchover export --out export/ --profile admin \
  --exclude '^/governance/roles/platform/terraform-' --exclude '^/governance/auth/ci-terraform$'
```

Writes `model/roles/imported.yaml`, `model/auth_methods.imported.yaml`, `model/folders.imported.yaml`, `model/governance.imported.yaml` and `imports.tf` with the `for_each` addresses of `terraform/*.tf`.

HCL mode: plain Terraform written by Terraform itself, for anything, including PKI issuers, account settings and auth-method types the YAML model doesn't cover.

```
export AKEYLESS_ACCESS_ID=p-... AKEYLESS_ACCESS_KEY=...       # read-only identity; provider env
↪  login
scripts/watchover export --out export/ --profile admin --hcl --generate
```

Writes `hcl/providers.tf` and `hcl/imports.tf` (one flat import block per object). With `--generate` it then runs `terraform init` and `terraform plan -generate-config-out=generated.tf`: the provider reads every object and Terraform writes its `resource` blocks into `hcl/generated.tf`. Without `--generate`, run those two commands yourself in `hcl/`.

17.4 Filters

Need	Flags
Only some families	`--types roles,auth-methods`
Only some names	‘--only 'sso
Part of the folder tree	`--root /payments --max-depth 5`
One user's view: their auth method and every role they reach	`--persona-auth-method /governance/auth/sso-entra-oidc` `--persona-claim email=alice@example.com` `--persona-claim groups=grp-akl-payments-admins`
Offline replay	`--from-dir export/raw` (raw/ of an earlier export)

The persona view uses the same sub-claim rules as the simulator (OR within a key, AND across keys, globs, case sensitivity per association). An auth method is configured per identity provider, not per user: ”the OIDC config for a user” is the OIDC auth method plus the associations whose sub-claims match that user's claims.

17.5 Secrets

- Exported files never contain secrets. In `raw/`, `client_secret`, access keys, passwords, private keys and tokens are replaced with `<redacted>`.
- OIDC auth methods get a `client_secret_var`, which you fill in `TF_VAR_auth_method_secrets`.
- In `generated.tf`, Terraform can't read back sensitive values. Replace those with variables before applying.
- After import, the state holds what the provider reads (e.g. API-key access keys). Treat the state as secret.

17.6 Procedure

1. Run the export and read `REPORT.md`. It lists everything that needs a decision: temporary rules (skipped), unknown rule types, root-scope roles, unsupported auth-method types, associations to objects outside the export, OIDC secrets to provide, issuers and settings that need `--hcl`.
2. Model mode: move the files into `terraform/model/` (merge with existing files, drop `.imported`) and copy `imports.tf` into `terraform/`. HCL mode: review `generated.tf`, then either keep the plain resources in their own state, or move them into the YAML model.
3. Run `terraform plan` until it shows **only imports**, with no create, destroy or replace. Then apply.
4. Delete `imports.tf`.

VERIFY:

- How `list-items` reports nested folders. The walk is breadth-first and bounded by `--max-depth`.
- Which `item_type` the API uses for PKI issuers. An item counts as an issuer when it has `pki_cert_issuer_details`, or when its type contains both PKI and ISSUER. `REPORT.md` lists the item types seen.

- That `import` blocks can't be exercised against a mocked provider (an OpenTofu limitation): in this repository they are validated, and their model plans are tested without them.

Chapter 18

CyberArk Conjur -> Akeyless

Two separate problems:

What	How
Secret values	Akeyless Gateway *Automatic Migration*, type `conjur` (static secrets only): `gateway.yaml -> migrations.conjur` (`akeyless_gateway_migration_conjur`) or `akeyless gateway-create-migration --type conjur ...`. Needs a Conjur identity that can read every variable; private CA -> make the bundle available on the Gateway and set `CONJUR_SSL_CERT_PATH`.
Authorization (policies, hosts, layers, groups, permits, grants)	Not migrated by Akeyless. `scripts/conjur2akeyless.py` translates Conjur policy YAML into model roles + a bindings report.

18.1 Translation rules

Conjur	Akeyless model
`!variable <policy>/<id>`	item `<item-prefix>/<policy>/<id>`
`execute` (fetch value)	`read` (+ `list`)
`read` (see metadata)	`list`
`update` (set value)	`update` (+ `list`)
a role with permits (layer, group, host, user, policy)	one role `<role-prefix>conjur/<kind>-<id>` with exact-path rules
`!grant role: A, member: B`	Akeyless has no role nesting -> B is bound to A's role too (transitive)
host / user identities	`bindings.yaml`: Akeyless roles each identity needs; you choose auth method + sub-claims
`!webservice`, `!deny`, `!revoke`, `!delete`, permits on non-variables	reported in `REPORT.md`, not translated

Anchored statement groups (`- &variables [ !variable a, ... ]` with `resources: *variables`) are supported. Annotations are ignored.

```
python3 scripts/conjur2akeyless.py root.yml prod.yml --item-prefix /conjur --role-prefix
↪  /governance/roles/ --out export/conjur
```

Mapping Conjur authenticators to Akeyless auth methods (decide per identity in `bindings.yaml`):

Conjur authenticator	Akeyless auth method	Typical sub-claims
authn-jwt (CI, GitHub/GitLab)	OAuth2/JWT	`repository`, `ref` / `project_path`
authn-k8s	Kubernetes	via Gateway K8s auth config; `bound_namespaces`, `bound_sa_names`
authn-iam	AWS IAM	`bound_aws_account_id`, `bound_role_name`
authn-azure	Azure AD	`bound_tenant_id`, `bound_spid`
authn-oidc / LDAP (humans)	OIDC / SAML / LDAP	`groups`
API key hosts	API key or Universal Identity (prefer an identity-based method)	-

Procedure: translate -> fill bindings as `associations` -> move `roles/conjur.yaml` into `terraform/model/roles/` -> tests + plan -> run the Gateway migration into the same prefix -> compare migrated item names with the generated rule paths (**VERIFY** the naming the migration produces) -> cut consumers over -> decommission Conjur policy.

Chapter 19

Certificates / PKI governance

- **Private CA signer**: RSA key with a CA certificate. `pki.yaml -> ca_keys` creates a DFC key with a self-signed certificate whose config carries `basicConstraints=critical, CA:TRUE` and `keyUsage=critical, keyCertSign, digitalSignature, cRLSign` (`create-dfc-key --generate-self-signed-certificate true --conf-file-path csr.conf`). An existing CA can be uploaded with `upload-rsa --cert` and referenced by name. Classic keys can sign only if Classic is enabled as protection-key type in account settings.
- **Issuer** = the certificate policy: allowed domains/SANs, wildcard/subdomain rules, EKU flags (`server_flag`, `client_flag`, `code_signing_flag`), key usage, subject fields, max TTL.
- `destination_path` stores issued certificates as items - required for `auto_renew` (+ `scheduled_renew_days`) and CRL. `protect_certificates` prevents their deletion.
- CRL: `create_public_crl` (served by Akeyless at `https://vault.akeyless.io/crl/<account-id>/<issuer-display-id>`) or `create_private_crl` (served by the Gateway; needs `gw_cluster_url`, port 8000). OCSP: `create_public_ocsp` / `create_private_ocsp`. ACME: `enable_acme` (needs `gw_cluster_url`).
- **Public CA** (Let's Encrypt, ZeroSSL, DigiCert, Sectigo, GlobalSign, GlobalSign Atlas, Google Trust...): create the CA target (`target create <ca>` / `akeyless_target_<ca>`), then an issuer with `ca_target` + `gw_cluster_url`. Sectigo approval workflows are not supported by the issuer.
- Expiry notifications: `expiration_event_in_days` on the issuer + an event forwarder with `certificate-pending-expiration` / `certificate-expired`.
- **RBAC**: issuers are items. Requesters need a rule on the issuer and on its destination path. The exact capability `get-pki-certificate` requires is not documented - **VERIFY** with `describe-permissions` (start with `read`; the reference model grants `read,list`).

Chapter 20

Security considerations

Governance as code moves the power to change access into a repository and a pipeline. Protect them as you protect the account itself.

20.1 Identities

- **The apply identity is administrator-equivalent.** Akeyless forbids granting rights you do not hold (anti-escalation), so the identity that manages all roles must hold them. It is bound to the repository *and* its protected default branch; protect that branch (required reviews, no force-push, no direct pushes) and gate the apply job with a protected environment.
- **The plan identity** is bound to the repository only, because it runs on merge requests. Keep its rights to reading; **VERIFY** its minimum read set on your account.
- **No static CI credentials.** The pipeline exchanges its own OIDC token (`watchover token`); nothing long-lived is stored in CI.
- **Bootstrap separately.** The CI identity lives in its own state so the main pipeline cannot destroy its own access. Delete the admin API key used for bootstrap, or keep it as a break-glass credential in your PAM vault.

20.2 Review

- CODEOWNERS on `terraform/model/roles/<bu>.yaml` (the business unit) and on everything else (the platform team). Require both for role files.
- Every merge request carries the plan; reviewers look for replacements, root-scope rules, removed deny rules and new associations without sub-claims.
- Access expectations make intent explicit: a reviewer can see the access matrix change.

20.3 Secrets and state

- Secret values never go into the YAML model; they are referenced by `*_var` keys and injected as sensitive `TF_VAR_*` maps from masked CI variables.
- Provider 2.0.5 has no write-only arguments, so the *state* contains OIDC client secrets, webhook tokens, Splunk tokens, the Conjur API key and computed private keys. Store it encrypted, versioned and readable only by the pipeline identities and a few administrators.
- `watchover export` redacts secrets in `raw/`, but exported files still describe your security model: keep export directories out of Git (`/export/` is ignored).

20.4 Guards and least privilege

The Terraform guards reject the most dangerous model mistakes before any plan reaches Akeyless: root-scope rules (`/*`) beyond list/deny without an explicit `allow_root_scope`, deny rules mixed with other capabilities, associations without sub-claims to auth methods that force them, associations to unmanaged auth methods, invalid management-plane values. Treat an `allow_*` flag in a merge request as a finding that needs a reason.

20.5 Working with Claude

- Do not paste secrets, tokens or state files into a conversation. The skill never needs them: it works on names, paths, claims and plans.
- Treat Claude's output as a proposal: it enters Akeyless only through the same tests, plan and review as any human change. Statements marked **VERIFY** are not confirmed by a primary source.
- Exported `REPORT.md` and plans can contain internal names and e-mail addresses; share them according to your data-handling rules.

20.6 The demo back end

`docs/manual/demo/akeyless` exists only to execute this manual's examples. It is not a client of Akeyless, answers from test fixtures, and must never be on the `PATH` of a real pipeline.

Chapter 21

Troubleshooting and FAQ

21.1 Guard failures (plan stops with “Resource precondition failed”)

The message names the guard. Fix the model; never disable the guard to make a plan pass.

Message starts with	Cause and fix
Duplicate role name	The same role key appears in two `roles/*.yaml` files. Keep one.
Object names must be absolute	A name lacks the leading `/` or ends with `/`.
Role names must start with / Auth method names must start with	Naming policy (`var.naming`). Rename, or set the prefix to `""` to disable the check.
Unknown auth method type	Supported in the model: api_key, oidc, saml, oauth2, aws_iam, azure_ad, k8s, universal_identity. Others: manage with `export --hcl`.
Unsupported rule type	Use item-, target-, role-, auth-method- or sra-rule; `ara-rule` needs `allow_ara_rules = true` after you verified it.
Invalid capability for the rule type	e.g. `allow_access` on an item-rule. See the list in the message.
A deny rule must contain only ’deny’	Split it: one rule with `[deny]`, another with the allowed capabilities.
approval_authority cannot be combined	Put approvers in a separate rule or role.
Root-scope rule	`/*` with more than list/deny. Narrow the path, or set `allow_root_scope: true` with a documented reason.
Invalid management-plane value	e.g. `audit_access: some`. Allowed values are in the model reference.
Association references an auth method that is not in	Add the auth method to the model, or set `allow_external_auth_methods: true` on the role.
Auth method has force_sub_claims = true	Every association to it needs `sub_claims`.
PKI issuer: …	Exactly one of `signer_key_name` / `ca_target`; `auto_renew` needs `destination_path`; CRL/OCSP/ACME/public CA need `gw_cluster_url`.
custom_fields.object_type	STATIC_SECRET, DYNAMIC_SECRET or ROTATED_SECRET.
event_forwarders: type must be	Type or event name not valid; `watchover forwarder` validates both.
event_forwarders: runner_type periodic	Add `every` (hours).
event_forwarders: slack and teams need url_var	The webhook URL is a secret: add `url_var` and the key to `TF_VAR_event_forwarder_secrets`.
event_forwarders of type teams require	Add `gateway_event_source_locations` and `runner_type`.
model/gateway.yaml defines gateway objects	Set `gateway_api` (CI variable `AKEYLESS_GATEWAY_API`).
gateway allowed_access: …	Unknown permission, or both/neither of `auth_method` and `access_id`.
gateway log_forwarding.type	syslog or splunk; other targets need an extra resource block.

21.2 Pipeline and Terraform

`argument "jwt" is required`
`TF_VAR_pipeline_jwt` is not exported in the job. The provider's environment fallback is not enough; the variable must be set.

Authentication fails (401) in CI
The token's claims do not match the role binding: wrong repository/project path, a branch other than the protected default branch for apply, or an audience other than `akeyless`. Compare the token's claims with `bootstrap.tfvars`.

Plan on a merge request fails with an authorization error
The plan role lacks a read right (**VERIFY** item). Extend `bootstrap/` for the object family named in the error.

Plan wants to *replace* a role or an auth method
Its name (YAML key) changed. Names are ForceNew; see "Rename a role or an auth method in Chapter 4".

Plan shows changes right after an apply
A value is normalised by Akeyless (case, ordering, defaults). Write it in the model the way Akeyless returns it; `export` shows that form.

Nightly drift job fails
Someone changed Akeyless outside Git; see "Handle a drift finding".

OpenTofu panics in a test with import blocks
Known OpenTofu behaviour with mocked providers; the kit validates import blocks but plans the imported model without them.

21.3 Kit commands

`watchover test: test_usage_guide_is_generated_and_current` fails
A command or option changed. Run `scripts/watchover help --markdown > docs/usage-guide.md` and `python3 docs/manual/build.py`.

`verify`: "describe-permissions …failed"
Check that the profile works (`akeyless list-roles --profile <p>`). If the call fails only for paths the persona cannot access, your CLI reports "no access" as an error: use `--error-means-no-access`.

`verify`: everything skipped
No persona has a `profile:` key.

`export`: a type is missing
`gateway` is not in the default `--types`; event forwarders cannot be listed by Akeyless (use `watchover forwarder`).

`export --transport api`: 401
The token expired or is missing: pass `--token` or set `AKEYLESS_TOKEN`; tokens from `watchover token` are short-lived.

`token`: "authentication failed"
Wrong access ID, an expired JWT, or claims that do not match the CI auth method's bound claims.

`account-settings` locked you out
A `bound-ips` value excluded the CI runners or your network. Fix it as an administrator from an allowed network; always `--dry-run` first.

`forwarder --write`: "already exists"
The name is in `event_forwarders`; edit it instead, or choose another name.

21.4 Frequently asked questions

Can I edit roles in the console?
You can, but the next apply reverts it. Change Git.

Why YAML and not plain HCL?
Reviewers read access, not Terraform; the guards and the simulator need a uniform model; the HCL stays generic and tested.

Can I use plain HCL for objects outside the model?
Yes: `export --hcl --generate` writes plain resources; keep them next to the model in the same root module.

API or CLI?
Both reach the same operations (Chapter 14). Use the CLI on workstations and the REST API in locked-down runners.

Several Gateway clusters?
One state or workspace per cluster, each with its own `gateway_api` and `model/gateway.yaml`.

Does the skill need access to my account?
No. It produces files and commands; you run them.

Chapter 22

Maintaining and extending the kit

22.1 Updating the kit in your governance repository

The kit is copied, not vendored, so updates are deliberate:

1. Read `CHANGELOG.md` of the new version.
2. Copy the new `terraform/*.tf`, `terraform/tests/`, `scripts/` and CI template over yours; do *not* overwrite `terraform/model/`.
3. Run `scripts/watchover test`, then open a merge request. The plan of a kit update should be empty; anything else needs an explanation (usually a new optional attribute).

22.2 Upgrading the Terraform provider

The HCL relies on documented provider facts (Chapter 11, "Provider facts"), checked against v2.0.5. Before raising the version constraint: re-check each fact in the new provider's schema and changelog, run the tests against the new version, and review a plan against a non-production account if you have one. Watch especially for newly ForceNew attributes and for changed defaults.

22.3 Versioning and releases

The kit and the skill share one version (`VERSION` in `scripts/watchover.py`), recorded in `CHANGELOG.md`. CI (GitHub Actions, `.github/workflows/test.yml`) runs the Terraform suites on both HashiCorp Terraform and OpenTofu, then the Python tests and the simulator, then packages the skill as an artefact.

22.4 Adding a kit command

1. Write `scripts/<name>.py` with `build_parser()` and `main(argv)`; every option needs a `help` text (a test enforces it).
2. Register it in `TOOLS` in `scripts/watchover.py` with a summary and usage patterns.
3. Add tests to `scripts/tests/test_scripts.py`.
4. Regenerate the usage guide (`scripts/watchover help --markdown > docs/usage-guide.md`).
5. Add an example execution to `examples()` in `docs/manual/build.py` (the build fails when a command has none) and rebuild the manual.

Adding a skill command: add it to `SKILL_COMMANDS` and to the command table in `skill/akeyless-watchover/SKILL.md` (a test checks they match), and write its example session in `docs/manual/sessions/<name>.tex`.

22.5 Adding a guard

Add a precondition to `terraform_data.model_guards` in `terraform/main.tf`, a negative fixture under `terraform/tests/fixtures/<name>/`, a `run` with `expect_failures` in `tests/model.tftest.hcl`, and the expected message to `tests/guard_messages.sh` so the test proves the *intended* guard fired. Add the message to the troubleshooting table (Chapter 21).

22.6 Maintaining this manual

Content	Source (edit here)
Introduction, installation, quick start, workflows, security, troubleshooting, maintenance, appendices	`docs/manual/chapters/*.tex`
Skill command example sessions	`docs/manual/sessions/*.tex`
Command reference (all options, usage patterns)	`scripts/watchover.py` and each script's `build_parser()`
Example executions	`examples()` in `docs/manual/build.py` (executed at build time)
Terraform variables and outputs	`variable`/`output` blocks in `terraform/` and `bootstrap/`
Concept, reference and operations chapters	`docs/*.md` (converted with pandoc)
Layout, title page, chapter order	`docs/manual/manual.tex`

`python3 docs/manual/build.py` regenerates `docs/manual/generated/`, executes the examples and writes `docs/akeyless-watchover-manual.pdf`. A Python test runs `build.py --check` (when pandoc is installed) so the generated reference parts cannot fall behind the code; rebuild the PDF whenever you change commands, options, variables or documents.

Chapter 23

Limitations, workarounds and open verifications

23.1 Not possible (platform or provider 2.0.5)

Limitation	Consequence	What this repo does
No per-rule resource; `akeyless_role.rules` authoritative	all rules of a role in one place; console edits reverted	YAML merge per role; nightly drift job
Names are ForceNew everywhere	rename = destroy + create; auth methods get a new access ID	`prevent_destroy`, delete protection, naming guards; CLI `update-role --new-name` for roles
Temporary rules (`--ttl`) CLI/API-only	removed by the next apply on that role	exporter skips them; use access requests for JIT
Partial `akeyless_account_settings`, destroy is a no-op	bound IPs, locks, client types, rotation limits, usage events not in TF	`account_settings_cli` + `akl_account_settings.py` in the apply job
ARA: no `ara_reports_access`; `ara-rule` undocumented	-	`allow_ara_rules` opt-in after verification
No write-only/ephemeral arguments	secrets in state	secret values kept out of this state; state treated as a vault
`akeyless_folder` has no `type/accessibility`	-	item folders only
No RBAC users/groups	human scope = IdP claims	`force_sub_claims`, CODEOWNERS on role files
`deny` does not apply to SRA	-	use SRA capabilities
K8s/LDAP auth need Gateway-side config	-	out of scope (Gateway)
Conjur RBAC not migrated by Akeyless	-	`conjur2akeyless.py`
One Gateway per provider alias	-	one state/workspace per Gateway cluster

23.2 Not verified against a live account (marked VERIFY)

- Minimum read rights of the plan identity (`bootstrap/`).
- Capability needed to issue a certificate from an issuer.
- `ara-rule` through the provider.
- Sub-claim template syntax `{{x}}` vs `{x}`.
- `/a/*` matching `/a` itself; operator evaluation order (simulator assumptions).
- `access_info.rules_type` strings (exporter); exact CLI behaviour of `describe-permissions` when the caller has no access (verifier flag).

- Item names produced by the Conjur migration.
- Export: `target_type` strings other than aws, azure, gcp, k8s and hashi_vault (matched by name); the CLI command name `list-groups` (REST `/list-group`); the shape of `admins.allowed_access` and the log-forwarding type in `gateway-get-config`; `/configure` and `/connect` over REST.

Everything else in this repository was checked against docs.akeyless.io, the Akeyless OpenAPI/Go SDK and the provider source/schema, and the code is covered by offline tests.

Appendix A

Exit codes

Command	Exit codes
all kit commands	0 success; 2 usage error (unknown command or option, invalid input)
`test`	0 all checks passed; otherwise the exit code of the first failing step
`simulate`	0 all expectations met; 1 at least one mismatch; 2 model or expectations error
`verify`	0 all checked expectations met; 1 at least one mismatch; 2 a CLI call failed
`export`	0 written (read `REPORT.md`); 2 a failed read, an HTTP error or invalid options
`forwarder`	0 written; 2 invalid combination or duplicate name (nothing is written)
`conjur`	0 written; 2 unreadable or invalid policy file
`account-settings`	2 invalid settings; otherwise the exit code of `akeyless update-account-settings`; 0 with `--dry-run`
`token`	0 token printed; 1 missing settings or authentication failed
`build-skill`	0 ZIP written
CI drift job	`plan -detailed-exitcode`: 0 no changes, 2 drift (job fails), 1 error

Appendix B

Environment variables

Variable	Used by and meaning
`AKEYLESS_API`	`token`, `export --transport api`: API base URL (default `https://api.akeyless.io`, or a Gateway `/api/v2`)
`AKEYLESS_ACCESS_ID`	`token`: access ID of the CI auth method. Provider and `export --hcl --generate`: with `AKEYLESS_ACCESS_KEY`, API-key login
`AKEYLESS_ACCESS_KEY`	provider API-key login (bootstrap, `--generate`); never in CI for the main pipeline
`CI_JWT`	`token`: the CI-issued JWT
`AKEYLESS_TOKEN`	`export --transport api`, `account-settings`: Akeyless token when `--token` is not given
`AKEYLESS_GATEWAY`	provider fallback for the API endpoint
`TF_VAR_pipeline_jwt`	Terraform: the CI JWT (must be set; see troubleshooting)
`TF_VAR_pipeline_access_id`	Terraform: the CI auth method's access ID (CI variable `AKEYLESS_PIPELINE_ACCESS_ID`)
`TF_VAR_auth_method_secrets`	JSON map for `*_var` keys in `auth_methods.yaml`
`TF_VAR_event_forwarder_secrets`	JSON map for `*_var` keys of event forwarders
`TF_VAR_gateway_secrets`	JSON map for `*_var` keys in `gateway.yaml`
`TF_VAR_gateway_api`	Gateway V2 API for gateway objects (CI variable `AKEYLESS_GATEWAY_API`)

Appendix C

Repository layout

```
terraform/            data-driven Terraform (provider akeyless-community/akeyless ~> 2.0)
  versions.tf         provider pin, JWT login, gateway alias, endpoint variables
  main.tf             inputs, model loading, guards, folders
  auth_methods.tf roles.tf governance.tf pki.tf gateway.tf outputs.tf
  model/              THE MODEL: roles/<bu>.yaml, auth_methods, folders, governance, pki,
                      gateway, access_expectations
  tests/              terraform test (mocked provider), fixtures/, guard_messages.sh,
                      export_roundtrip.sh
bootstrap/            one-time CI identity (OAuth2/JWT) + plan/apply roles, own state
scripts/              watchover (entry point) and the tools behind it
  tests/              pytest suite, fake CLI/REST back ends, fixtures
ci/                   GitHub Actions and GitLab CI templates, CODEOWNERS example
skill/                SKILL.md source of the Claude skill
docs/                 Markdown documentation (bundled in the skill), this PDF
  manual/             LaTeX sources of this manual, build.py, demo back end
```

Appendix D

Glossary

Access role
An Akeyless role: a set of rules (path + capabilities) plus management-plane permissions.

Association
The binding of a role to an auth method, optionally narrowed by sub-claims.

Auth method
How an identity authenticates (OIDC, SAML, AWS IAM, Kubernetes, API key, …); it carries an access ID.

Anti-escalation
Akeyless refuses to let an identity grant rights it does not hold itself.

Authoritative
Terraform owns the complete list: rules not in the model are removed on apply.

Capability
An allowed action on a path: read, list, create, update, delete, deny, or SRA capabilities.

Drift
A difference between the account and Git, detected by a non-empty plan.

ForceNew
A provider attribute whose change replaces the object (destroy + create).

Guard
A Terraform precondition that rejects an invalid or unsafe model before any change.

Management plane
Who may administer Akeyless itself (audit, analytics, gateways, event centre …), as opposed to accessing secrets.

Model
The YAML files under `terraform/model/`: the single source of truth.

Persona
A test identity (auth method + claims) used in access expectations.

Rule type
What a rule applies to: items, targets, roles, auth methods, SRA, ARA.

SRA
Secure Remote Access: brokered sessions to infrastructure.

Sub-claims
Claim values an identity must present for an association to apply (OR within a key, AND across keys).

VERIFY
A statement not confirmed by a primary source; check it on your account.

Appendix E

Sources

- Akeyless documentation: `https://docs.akeyless.io/`
- Akeyless OpenAPI specification and Go SDK v5.0.x (the API operations and their request schemas)
- Terraform provider `akeyless-community/akeyless` v2.0.5: source code and schema (`https://registry.terraform.io/providers/akeyless-community/akeyless/latest/docs`)
- Terraform and OpenTofu documentation for `terraform test`, mocked providers and import blocks

Statements that no primary source confirms are marked **VERIFY** (Chapter 23).